

WEB TECHNOLOGIES

HTML 5 – Audio, Video and Progress elements

Prof. Vinay Joshi and Dr. Sarasvathi V

Department of Computer Science and Engineering

HTML5 - Audio

Audio elements

- A standard approach for browser to play audio without the use of plug-in.
- Mp3, Wav and ogg formats supported by HTML5

Syntax

```
<audio controls="c1">
  <source src="song.ogg" type="audio/ogg" />
  <source src="song.mp3" type="audio/mp3" />
  Your browser does not support the audio element.
</audio>
```

HTML5 - Audio

Audio element properties

Attribute	Value	Description
autoplay	autoplay	Specifies that the audio will start playing as soon as it is ready.
controls	controls	Specifies that controls will be displayed, such as a play button.
loop	loop	Specifies that the audio will start playing again (looping) when it reaches the end
preload	preload	Specifies that the audio will be loaded at page load, and ready to run. Ignored if autoplay is present.
src	url	Specifies the URL of the audio to play

HTML5 - Video

Video elements

- A standard approach for browser to play video without the use of plug-in

Syntax

```
<video width="320" height="240" controls="controls">
  <source src="movie.ogg" type="video/ogg" />
  <source src="movie.mp4" type="video/mp4" />
  <source src="movie.webm" type="video/webm" />
  Your browser does not support the video tag.
</video>
```

HTML5 - Video

Video element properties



Attribute	Value	Description
audio	muted	Defining the default state of the the audio. Currently, only "muted" is allowed
autoplay	autoplay	If present, then the video will start playing as soon as it is ready
controls	controls	If present, controls will be displayed, such as a play button
height	<i>pixels</i>	Sets the height of the video player
loop	loop	If present, the video will start over again, every time it is finished
poster	<i>url</i>	Specifies the URL of an image representing the video
preload	preload	If present, the video will be loaded at page load, and ready to run. Ignored if "autoplay" is present
src	<i>url</i>	The URL of the video to play
width	<i>pixels</i>	Sets the width of the video player

HTML5 - Video

Progress elements



- The <progress> tag represents the completion progress of a task.
- Always add the <label> tag for describing the task!
- Use JavaScript to manipulate the value of the progress bar

Syntax:

<label for="file">Downloading progress:</label>

<progress id="file" value="32" max="100"> 32% </progress>

Attribute	Value	Description
max	<i>number</i>	Specifies how much work the task requires in total. Default value is 1
value	<i>number</i>	Specifies how much of the task has been completed

Unit 2: HTML5, JQuery and Ajax

HTML5 <audio> tag

It is used to play audio in html pages. It takes the below basic format in its simplest form:

```
<audio src="my_music.mp3" controls></audio>
```

With the above structure, when the html page loads the page requests for the audio file listed in the "src" attribute and the "controls" attribute displays the browser default audio player for controlling playback.

CODE:

```
<!doctype html>·
<html lang="en">·
<head>·
...·<meta charset="UTF-8">
...·<title>Intro to Audio</title>
</head>
<body>·
...·<audio src="sample.mp3" controls></audio>·
</body>·
</html>·
```

Results:



The <audio> tag comes with many inline global attributes that help in modifying its behaviour. Some of the attributes are:

1. autoplay
2. buffered

3. controls
4. loop
5. muted
6. played
7. preload
8. src
9. Volume

Some of the attributes, values and description are given below:

Attribute	Value	Description
autoplay	autoplay	Specifies that the audio will start playing as soon as it is ready.
controls	controls	Specifies that controls will be displayed, such as a play button.
loop	loop	Specifies that the audio will start playing again (looping) when it reaches the end
preload	preload	Specifies that the audio will be loaded at page load, and ready to run. Ignored if autoplay is present.
src	url	Specifies the URL of the audio to play

HTML 5 <video> Tag

The HTML 5 <video> tag is used to specify video on an HTML document. For example, it can be embed in a music video on web page for visitors to listen to and watch. The <video> tag was introduced in HTML 5.

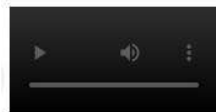
The HTML 5 <video> tag accepts attributes that specify how the video should be played. Attributes include preload, autoplay, loop and more.

Any content between the opening and closing <video> tags is fallback content. This content is displayed only by browsers that don't support the <video> tag.

Attribute	Value	Description
audio	muted	Defining the default state of the the audio. Currently, only "muted" is allowed
autoplay	autoplay	If present, then the video will start playing as soon as it is ready
controls	controls	If present, controls will be displayed, such as a play button
height	pixels	Sets the height of the video player
loop	loop	If present, the video will start over again, every time it is finished
poster	url	Specifies the URL of an image representing the video
preload	preload	If present, the video will be loaded at page load, and ready to run. Ignored if "autoplay" is present
src	url	The URL of the video to play
width	pixels	Sets the width of the video player

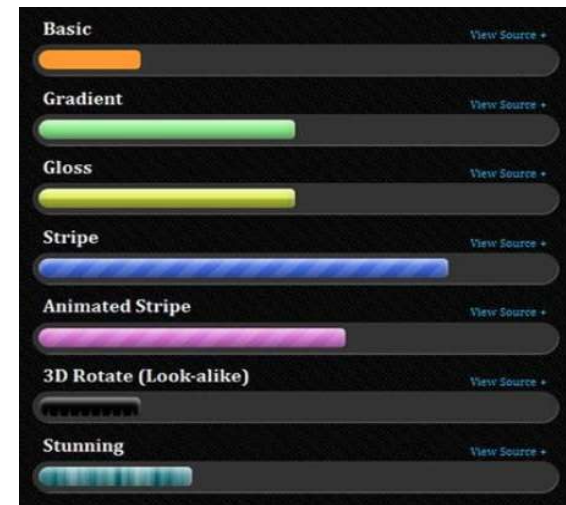
CODE:

```
<video src="/video/pass-countdown.ogg" width="170" height="85" controls>
<p>If you are reading this, it is because your browser does not support the HTML5 video element.</p>
</video>
```

Results:**HTML 5 <progress> Tag**

The <progress> element is used to create a progress bar to serve as a visual demonstration of progress towards the completion of task or goal. The max and value attributes are used to define how much progress (value) has been made towards task completion (max).

It can be indeterminate progress bar, which can be either in the form of spinning wheel or a horizontal bar. In this mode, the bar only shows cyclic movements and do not provide the exact progress indication. This mode is usually used at the time when the length of the time is not known.

**CODE:**

```
<progress value="33" max="100"></progress>
```

Results:

Reference Link for styling progress elements:

<https://css-tricks.com/html5-progress-element/>

WEB TECHNOLOGIES

HTML 5 – Canvas & SVG

Prof.Vinay Joshi and Dr.Sarasvathi V
Department of
Computer Science and Engineering

HTML5 - Canvas Canvas element

- Uses JavaScript to draw graphics on a web page
- A rectangular area, and you control every pixel of it

Syntax

```
<canvas id="myCanvas" width="200" height="100">  
  Canvas is not supported  
</canvas>
```

HTML5 - Canvas Canvas – context object

- The canvas element has no drawing abilities of its own. All drawing must be done inside a JavaScript using the context object

```
<script type="text/javascript">  
  var c=document.getElementById("#myCanvas");  
  var ctx=c.getContext("2d");  
  ctx.fillStyle="#FF0000";  
  ctx.fillRect(50,50,150,75);  
</script>
```



HTML5 - Canvas Canvas – context methods

Method	Description
fillRect(x, y, width, height)	Draws a filled rectangle
strokeRect(x, y, width, height)	Draws a rectangular outline
clearRect(x, y, width, height)	Clears the specified rectangular area, making it fully transparent
moveTo(x, y)	Moves the pen to the coordinates specified by x and y
lineTo(x, y)	Draws a line from the current drawing position to the position specified by x and y
arc(x, y, r, sAngle, eAngle, anticlockwise)	Draws an arc centered at (x, y) with radius r starting at sAngle and ending at eAngle going anticlockwise (defaulting to clockwise).
arcTo(x1, y1, x2, y2, radius)	Draws an arc with the given control points and radius, connected to the previous point by a straight line

HTML5 - Canvas

Canvas – context methods (cntd.)

Method	Description
<code>createLinearGradient(x1, y1, x2, y2)</code>	Creates a linear gradient object with a starting point of (x1, y1) and an end point of (x2, y2).
<code>createRadialGradient(x1, y1, r1, x2, y2, r2)</code>	Creates a radial gradient. The parameters represent two circles, one with its center at (x1, y1) and a radius of r1, and the other with its center at (x2, y2) with a radius of r2.
<code>fillText(text, x, y [, maxWidth])</code>	Fills a given text at the given (x,y) position. Optionally with a maximum width to draw.
<code>strokeText(text, x, y [, maxWidth])</code>	Strokes a given text at the given (x,y) position. Optionally with a maximum width to draw.
<code>drawImage(image, x, y [,width, height])</code>	Draws the CanvasImageSource specified by the image parameter at the coordinates (x, y) with optional width and height

HTML5 – SVG

SVG – Predefined Shape Element

- `<rect width="300" height="100" style = "fill:rgb(0,0,255); stroke-width:3; stroke:rgb(0,0,0)" />`
- `<circle cx="50" cy="50" r="40" stroke="black" stroke-width="3" fill="red" />`
- `<polygon points="200 100,300 100,300 200,200 200" style = "fill:lime; stroke:purple; stroke-width:1" />`
- `<ellipse cx="200" cy="80" rx="20" ry="50" style = "fill:yellow; stroke:purple; stroke-width:2" />`
- `<text x="0" y="15" fill="red" transform="rotate(30 20,40)">I love SVG </text>`



HTML5 – SVG

SVG Element

- SVG stands for Scalable Vector Graphics.
- SVG defines vector-based graphics using HTML elements
- SVG graphics do NOT lose any quality if they are zoomed or resized

```
<svg width="100" height="100">  
  <circle cx="50" cy="50" r="40" stroke="green" stroke-width="4"  
    fill="yellow" />  
</svg>
```



Unit 2: HTML5, JQuery and Ajax

HTML 5 <canvas> Tag

The Canvas API provides a means for drawing graphics via JavaScript and the HTML <canvas> element. It can be used for animation, game graphics, data visualization, photo manipulation, and real-time video processing. Canvas allows you to render graphics powered by JavaScript. Some of the Canvas context methods are following:

Method	Description
fillRect(x, y, width, height)	Draws a filled rectangle
strokeRect(x, y, width, height)	Draws a rectangular outline
clearRect(x, y, width, height)	Clears the specified rectangular area, making it fully transparent
moveTo(x, y)	Moves the pen to the coordinates specified by x and y
lineTo(x, y)	Draws a line from the current drawing position to the position specified by x and y
arc(x, y, r, sAngle, eAngle, anticlockwise)	Draws an arc centered at (x, y) with radius r starting at sAngle and ending at eAngle going anticlockwise (defaulting to clockwise).
arcTo(x1, y1, x2, y2, radius)	Draws an arc with the given control points and radius, connected to the previous point by a straight line

CODE:

```
<p>Before canvas.</p>
<canvas width="120" height="60"></canvas>
<p>After canvas.</p>
<script>
  let canvas = document.querySelector("canvas");
  let context = canvas.getContext("2d");
  context.fillStyle = "red";
  context.fillRect(10, 10, 100, 50);
</script>
```

Results:

Before canvas.



After canvas.

HTML <svg>Tag

The svg element is a container that defines a new coordinate system and viewport. It is used as the outermost element of SVG documents, but it can also be used to embed a SVG fragment inside an SVG or HTML document.

The xmlns attribute changes an element (and its children) to a different XML namespace. This namespace, identified by a URL, specifies the dialect that we are currently speaking. The <circle> and <rect> tags, which do not exist in HTML, do have a meaning in SVG—they draw shapes using the style and position specified by their attributes.

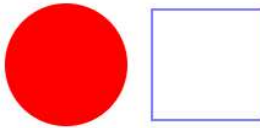
These tags create DOM elements, just like HTML tags, that scripts can interact with.

CODE:

```
<p>Normal HTML here.</p>
<svg xmlns="http://www.w3.org/2000/svg">
  <circle r="50" cx="50" cy="50" fill="red"/>
  <rect x="120" y="5" width="90" height="90"
  ... stroke="blue" fill="none"/>
</svg>
```

Results:

Normal HTML here.



HTML5 – Geo Location

Introduction

- Enables your web application to obtain the geographical position of your website visitors
- The user has to accept to share their location,
- The Geolocation API is work with the navigator.geolocation object.
- Accessed via JavaScript, through the **navigator.geolocation** object.



HTML5 – Geo Location

Methods

- navigator.geolocation object allows you to access geo location through two primary functions:
 - getCurrentPosition()
 - Returns the location of the visitor as a one-time snapshot
 - watchPosition()
 - returns the location of the visitor every time the location changes

Syntax : getCurrentPosition(success, error, options)

- Both functions take the following parameters:
 - Success callback function
 - Error callback function (optional)
 - Geo location options object (optional)
 - If the getCurrentPosition() method is successful, it returns a coordinates object to the function specified in the parameter (success)

HTML5 – Geo Location

Introduction

The Geolocation API uses three methods of Geolocation interface which are given following:

Methods	Description
getCurrentPosition()	It identifies the device or the user's current location and returns a position object with data.
watchPosition()	Return a value whenever the device location changes.
clearWatch()	It cancels the previous watchPosition() call



HTML5 – Geo Location

Code Example

```
navigator.geolocation.getCurrentPosition(showPosition);  
function showPosition(position) {  
    x.innerHTML = "Latitude: " + position.coords.latitude +  
    "<br>Longitude: " + position.coords.longitude;  
}
```

```
navigator.geolocation.getCurrentPosition( function(position) {  
    console.log("your position is: " + position.coords.latitude + " , " +  
    position.coords.longitude);  
} );
```

* getCurrentPosition can be replaced with watchPosition if you need to receive updates to location



Unit 2: HTML5, JQuery and Ajax

HTML5 Geolocation API:

The Geolocation API of HTML5 helps in identifying the user's location, which can be used to provide location specific information or route navigation details to the user. There are many techniques used to identify the location of the user. The Geolocation API protects the user's privacy by mandating that the user permission should be sought and obtained before sending the location information of the user to any website. So the user will be prompted with a popover or dialog requesting for the user's permission to share the location information. The user can accept or deny the request.

The current location of the user can be obtained using the `getCurrentPosition` function of the `navigator.geolocation` object. This function accepts three parameters – Success callback function, Error callback function and position options. If the location data is fetched successfully, the success callback function will be invoked with the obtained position object as its input parameter. Otherwise, the error callback function will be invoked with the error object as its input parameter.

jQuery

Introduction

- Fast and concise
- Simplifies the interaction between HTML and JavaScript
- Lightweight : 19KB in size (Minified and Gzipped)
- CSS1 - 3 Complaint
- Cross Browser
 - (IE 6.0+, FF 2+, Safari 3.0+, Opera 9.0+, Chrome)
- It makes things like HTML document traversal and manipulation, event handling, animation, and Ajax



jQuery

Write less code, to do more

Vanilla JavaScript:

```
let paras = document.querySelectorAll("p")
for (let i=0; i<paras.length; i++)
    paras[i].style.color = "red"
```

jQuery:

```
$("p").css("color", "red");
```



jQuery

Including the Library

- Download jQuery from http://docs.jquery.com/Downloading_jQuery
- Include the library in your web page

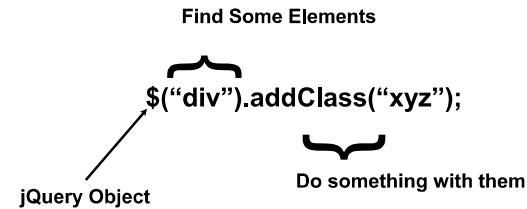
```
<head>
<script src="path/to/jquery-x.x.js"></script>
</head>
```
- OR
- Include from a CDN (Content Delivery Network)

```
<head>
<script src = "https://code.jquery.com/jquery-3.5.1.min.js"> </script>
</head>
```



jQuery

Statement Structure



Select By	Example
ID	<code>\$("#header")</code>
Class	<code>\$(".updated")</code>
Tag Name	<code>\$("table")</code>
Combination	<code>\$("table.user-list")</code> or <code>\$("#footer ul.menu li")</code>
Basic Filters	<code>:first</code> , <code>:last</code> , <code>:even</code> , <code>:odd</code>
Content Filters	<code>:empty</code> , <code>:contains(text)</code> , <code>:has(selector)</code>
Attribute Filters	<code>[attribute]</code> , <code>[attribute=value]</code> , <code>[attribute!=value]</code>
Forms	<code>:input</code> , <code>:text</code> , <code>:submit</code> , <code>:password</code> , <code>:enabled</code> , <code>:checked</code>

Action	Example
DOM Manipulation	<code>before()</code> , <code>after()</code> , <code>append()</code> , <code>appendTo()</code>
Attributes	<code>addClass()</code> , <code>css()</code> , <code>attr()</code> , <code>html()</code> , <code>val()</code> , <code>text()</code>
Events	<code>click()</code> , <code>on()</code> , <code>bind()</code> , <code>unbind()</code> , <code>live()</code>
Effects	<code>hide()</code> , <code>fadeOut()</code> , <code>toggle()</code> , <code>animate()</code>
AJAX	<code>load()</code> , <code>get()</code> , <code>ajax()</code> , <code>post()</code> , <code>getJSON()</code>

- `append()` – adds a set of elements to the end of the children
 - `$("pelem").append($c1[, c2, ...])`
 - Eg: `$("ul").append(newli1,newli2)`
`// $c1, $c2, ... will be appended to child elements of $pelem`
 - Similar methods : `appendTo`, `prepend`
- `after()` - adds a set of elements after the specified element
 - `$("elem").after($e1[, $e2,...])`
 - Eg: `$("li:last").after(newli1)`
`// $e1, $e2 will be added after $elem under the same parent`
 - Similar methods : `insertAfter`, `before`, `insertBefore`

- Attribute Methods like `css()`, `attr()`, `val()`, `html()`, `text()` can be used for both setting and getting attributes
- Based on whether one or two arguments were passed
- Example:
 - For Setting
`$("p:last").css("color", "green");`
 - For Getting
`let pcolor = $("p:last").css("color")`

jQuery

Chaining Methods

- Most jQuery methods return jQuery object
- You can chain them together to perform multiple operations on the same elements

```
$("#deleted").addClass("red").fadeOut("slow");
```

- This will not work as val() returns a string

```
$(":button").val("Click Me").click(function(){...})
```



jQuery – Events

Event Methods

- Register an event handler for an element object passing a function reference to the event method

```
$("span#message").click(function(event){...})
```

; OR

```
$("span#message").on("click", function(event) {...});
```



jQuery – Events and Effects

The Action Part

Action	Example
Events	click(), bind(), unbind(), live()
Effects	hide(), fadeOut(), toggle(), animate()



jQuery – Events

Manually triggering Event

- Like with attribute methods, the event methods have a different meaning without arguments
- Without the function reference argument, the event methods are treated like a manual firing of event

```
$("span#message").click();
```



- Special event to register for handler once the document has been loaded completely is ready

```
$(document).ready(function(){  
    ...  
});
```

- within any event handler function **this** element refers to the element for which the handler is called

```
$( "p" ).click(function() {  
    var htmlString = $( this ).html();  
    ...  
}
```



- Lot of animation/styling effects can be accomplished using the effects methods like hide, show, toggle, fadeIn, fadeout etc.

```
$("#show-cart").click(function(){  
    $("#cart").slideToggle("slow");  
});
```

- Or, use the animate method to build custom animations

```
$("#showdown").click(function(){  
    $("#my-  
        div").animate({  
        width: "70%",  
        opacity: 0.4,  
        fontSize: "3em"  
    }, 1200 );  
..
```



THANK YOU

Vinay Joshi

Department of Computer Science and Engineering

vinayj@pes.edu

+91 80 2672 6622

Unit 2: HTML5, JQuery and Ajax

jQuery

jQuery is a JavaScript Library. jQuery simplifies JavaScript programming. jQuery is a lightweight, "write less, do more", and JavaScript library. The purpose of jQuery is to make it much easier to use JavaScript on website. jQuery was originally released in January 2006 at BarCampNYC by John Resig.

jQuery is a JavaScript library that allows web developers to add extra functionality to their websites. It is open source and provided for free under the MIT license. In recent years, jQuery has become the most popular JavaScript library used in web development.

With jQuery you select (query) HTML elements and perform "actions" on them. Basic syntax is:

\$(selector).action ()

- A \$ sign to define/ access jQuery.
- “\$()” access an element in current html document.

A (selector) to "query (or find)" HTML elements.

A jQuery action () to be performed on the element(s).

Examples:

- \$("p").hide() - hides all elements.
- \$("#test").hide() - hides the element with id="test".
- \$(".test").hide() - hides all elements with class="test".

JavaScript vs. jQuery

Let us try to understand the difference between JavaScript and jquery.

Example 1 - Hide an element with id "textbox"

```
//JavaScript

document.getElementById(textbox).style.display = "none";

//jQuery

$("#textbox).hide();
```

Example 2 - Create a <h1> tag with "my text"

```
//JavaScript

var h1 = document.CreateElement("h1");

h1.innerHTML = "my text";

document.getElementsByTagName(body)[0].appendChild(h1);

//jQuery

$(body).append( $("<h1/>").html("my text") );
```

The Document Ready Event:

```
$(document).ready(function){
```

```
// jQuery methods go here...
```

```
}
```

This is to prevent any jQuery code from running before the document is finished loading (is ready).

- Trying to hide an element that is not created yet.
- Trying to get the size of an image that is not loaded yet

Alternate Syntax :

```
$(function){

    // jQuery methods go here...

}
```

jQuery Selectors:

jQuery selectors allow you to select and manipulate HTML. With jQuery selectors you can find elements based on their id, classes, types, attributes, values of attributes and much more. It's based on the existing CSS Selectors and in addition, it has some own custom selectors. All type of selectors in jQuery, start with the dollar sign and parentheses: \$().

Types of jQuery selectors .

- Element selector Id
- (#) selector Class
- (.) selector

Element Selector:

The jQuery element selector selects elements based on their tag names.

Example:

```
$(document).ready(function()
{
    $("#button").click(function()
    {
        $("#p").hide();
    });
});
```

Id (#) Selector :

The jQuery #id selector uses the id attribute of an HTML tag to find the specific element.

Example :

```
$(document).ready(function()
{
    $("#button").click(function()
    {
        $("#test").hide();
    });
});
```

Class (.) Selector:

The jQuery class selector finds elements with a specific class.

Example :

```
$(document).ready(function()
{
    $("#button").click(function()
    {
        $(".test").hide();
    });
});
```

More jquery Selectors :

Syntax	Description
\$("#*")	Selects all elements
\$(this)	Selects the current HTML element
\$("#p.intro")	Selects all <p> elements with class="intro"
\$("#p:first")	Selects the first <p> element
\$("#ul li:first")	Selects the first element of the first
\$("#ul li:first-child")	Selects the first element of every
\$("#[href]")	Selects all elements with an href attribute
\$("#a[target='_blank']")	Selects all <a> elements with a target attribute value equal to "_blank"
\$("#tr:even")	Selects all even <tr> elements
\$("#tr:odd")	Selects all odd <tr> elements

jquery Effects :

There are 3 types of jQuery Effects and they are:

- jQuery hide()
- jQuery show()
- jQuery toggle()

Syntax:

\$(selector).hide(speed, callback);

\$(selector)show(speed, callback);

\$(selector).toggle(speed, callback);

Speed and callback are optional parameters

jQueryEvent Methods:

Mouse Events	Keyboard Events	Form Events	Document/Window Events
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

jQuery Terminology:

- The **jQuery function** refers to the global jQuery object or the \$ function depending on the context .
- A **jQuery object** the object returned by the jQuery function that often represents a group of elements.
- **Selected elements** refers to the DOM elements that you have selected for, most likely by some CSS selector passed to the jQuery function and possibly later filtered further.

jQuery Methods**1. DOM Manipulation**

- before(), after(), append(), appendTo()

Example: Move all paragraphs in div with id “contents”

```

$("#p").appendTo("#contents");
$("#h1").append(" Dom Manipulation");
<body>
<h1>jQuery Dom Manipulation</h1>
<div id="contents">
    <p>jQuery is good</p>
    <p>jQuery is better</p>
    <p>jQuery is the best</p>
</div> </body>

```

2. Attributes

- `css()`, `addClass()`, `attr()`, `html()`, `val()`

Example : Setting

```
$("img.logo").attr("align", "left");
$("p.copyright").html("&copy; 2009 ajaxray");
$("input#name").val("Spiderman");
```

Example : Getting

```
var allignment = $("img.logo").attr("align");
var copyright = $("p.copyright").html();
var username = $("input#name").val();
```

3. Events

- `click()`, `bind()`, `unbind()`, `live()`

Example: Binding all interactions on events.

```
$(document).ready(function(){
    $("#message").click(function(){
        $(this).hide(); })
    });
<span id="message" onclick="..."> blah blah </span>
```

4. Effects

- `hide()`, `fadeOut()`, `toggle()`, `animate()`

Example :When “show-cart” link clicked, slide up/down “cart” div.

```
$("#show-cart").click(function(){
    $("#cart").slideToggle("slow"); })
```

5. Ajax

- `load()`, `get()`, `ajax()`, `getJSON()`

Examples: Load a page in a container

```
$("#comments").load ("get_comments.php");
$("#comments").load
("get_comments.php", {max: 5});
```

jQuery Method Chaining

Chaining Methods, also known as Cascading, refers to repeatedly calling one method after another on an object, in one continuous line of code. This technique abounds in jQuery and other JavaScript libraries and it is even common in some JavaScript native methods.

Example:

```
$("#wrapper").fadeOut().html("Welcome, Sir").fadeIn();
or this:
str.replace("k", "R").toUpperCase().substr(0,4);
```

is not just pleasurable and convenient but also succinct and intelligible. It allows us to read code like a sentence, flowing gracefully across the page. It also frees us from the monotonous, blocky structures we usually construct.

jQuery chaining allows you to execute multiple methods in a single statement. By doing that, it removes the need for repeatedly finding the same element to execute code. It also makes the code more compact and readable.

To perform jQuery method chaining, you should append actions to one another. Usually each statement is run as a separate operation. Chaining in jQuery is used to link multiple statements together. A chained jQuery statement is executed as one operation. Therefore, it runs faster.

WEB TECHNOLOGIES

jQuery – Callback and Promises

Vinay Joshi

Department of
Computer Science and Engineering

jQuery – Callbacks

Introduction

Synchronous code runs in sequence. This means that each operation must wait for the previous one to complete before executing.

```
console.log('One');
console.log('Two');
console.log('Three');
// LOGS: 'One', 'Two', 'Three'
```

Asynchronous code runs in parallel. This means that an operation can occur while another one is still being processed.

```
console.log('One');
setTimeout(function(){
    console.log('Two')
}, 1000);
console.log('Three');
// Logs One Three two
```

jQuery – Callbacks

Introduction

- As seen in setInterval, setTimeout and addEventListener, a function accepts a function reference as an argument
- They will be called asynchronously based on timer or other events
- These function references are called **Callback functions**
- Example:
div.addEventListener("keypress", function(){ ... });

jQuery – Promises

Introduction

Callback Hell

```
firstFunction(args, function() {
    secondFunction(args, function() {
        thirdFunction(args, function() {
            // And so on...
        });
    });
});
```

```
$(document).ready(function() {
    ("li.g1").click(function() {
        $(this).fadeOut(1000, function() {
            $(this).fadeIn(2000, function() {
                $(this).click()
            });
        });
    });
});
```

Difficulties to read and maintain.

If there is an error in one function, all the other functions would also get affected, making error handling more complicated.

I Promise a Result!"

"Producing code" is code that can take some time

"Consuming code" is code that must wait for the result

A Promise is a JavaScript object that links producing code and consuming code

A JavaScript Promise object contains both the producing code and calls to the consuming code:



- Promise is a JavaScript object with a value that may not be available at the moment when the code line executes.
- These values are resolved at some point in the future.
- It allows you to write asynchronous code more synchronously.



```
let myPromise = new Promise(function(myResolve, myReject) {  
  // "Producing Code" (May take some time)  
  
  myResolve(); // when successful  
  myReject();  // when error  
});  
  
// "Consuming Code" (Must wait for a fulfilled Promise)  
myPromise.then(  
  function(value) { /* code if successful */ },  
  function(error) { /* code if some error */ }  
);
```



A Promise Object Properties

JavaScript Promise object can be:

- Pending
- Fulfilled
- Rejected

The Promise object supports two properties: **state** and **result**.

While a Promise object is "pending" (working), the result is undefined.

When a Promise object is "fulfilled", the result is a value.

When a Promise object is "rejected", the result is an error object.

You cannot access the Promise properties state and result.

You must use a Promise method to handle promises.



jQuery – Promises

Introduction



```
// Promise constructor
let promise = new Promise(function(resolve, reject) {
  const x = "apple";
  const y = "apple"

  if (x === y) {
    resolve();
  } else {
    reject();
  }
});

// Consuming the Promise
promise
  .then(function () {
    console.log('Successful');
  })
  .catch(function () {
    console.log('Some error has occurred');
  });
```

- Promise constructor takes only one argument which is a callback function (and that callback function is also referred as an anonymous function too).
- Callback function takes two arguments, resolve and reject. Perform operations inside the callback function and if everything went well then call resolve.

jQuery – Promises

Introduction



- A promise is used to handle the asynchronous result of an operation.
- With Promises, we can defer execution of a code block until an async request is completed.
- The Promise object is created using the new keyword and contains the promise; this is an executor function which has a **resolve** and a **reject** callback
- Essentially, a promise is a returned object to which you attach callbacks, instead of passing callbacks into a function.

```
const promise = new Promise(function(resolve, reject) {
  // promise description
});
```

jQuery – Promises

Example



```
var weather;
const date = new Promise(
  function(resolve, reject) {
    weather = true; //usually a API call
    if (weather) {
      const dateDetails = {
        name: 'Cubana Restaurant',
        location: '55th Street',
        table: 5
      };
      resolve(dateDetails)
    } else {
      reject(new Error('Bad weather'))
    }
  }
);
```

```
date
  .then(function(done) {
    console.log('We are going on a date!')
    console.log(done)
  })
  .catch(function(error) {
    console.log(error.message)
  })
```

jQuery – Callback and Promises

Comparison



- Callbacks and Promises are not the same
- Callbacks are function passed to another function as a reference
- Chaining of Callbacks can be clumsy and lead to **Callback Hell**
- Promises use Callbacks and more elegant than Callbacks
- Chaining of Promises is supported

Unit 2: HTML5, JQuery and Ajax

jquery: Callback

A callback function is executed after the current effect is 100% finished. JavaScript statements are executed line by line. However, with effects, the next line of code can be run even though the effect is not finished. This can create errors. To prevent this, you can create a callback function.

Syntax :

```
$(selector).hide(speed, callback);
```

Example : callback 1 .html

In JavaScript, statement lines are executed one by one. It might cause problems at times, as a certain effect might start running before the previous one finishes.

To prevent that, callback function jQuery comes in handy. It creates a queue of effects so they are run in a row.

CODE:

```
<!DOCTYPE html>
<html>
<head>
<script src="https://code.jquery.com/jquery-3.4.1.min.js"
  integrity="sha256-CSXorXvZcTkaix6Yv66HppcZGetbYMGWsf1Bw8HfCJo="
  crossorigin="anonymous"></script>
<script type="text/javascript" src="scripts.js"></script>
<link rel="stylesheet" href="styles.css">
</head>
<body>

<button>Click me</button>

<p>This is a paragraph make it dissappear</p>

</body>
</html>
```

```
$(document).ready(() => {
  $("button").click(() => {
    $("p").hide("slow", () => {
      alert("Congratulations it works");
    });
  });
});
```

Output:

Click me

This is a paragraph make it dissappear

Once you click on Click ME button the texts disappears and alert box pops up:

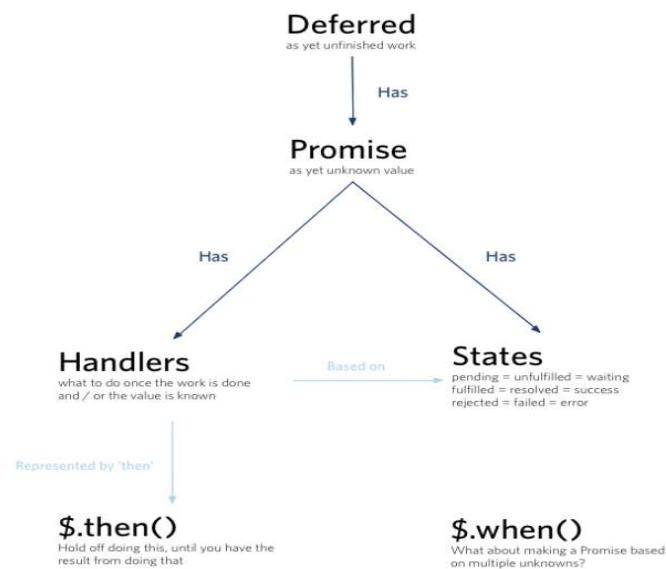


Callback functions are simple and get the job done, but they become unmanageable as soon as you need to execute many asynchronous operations, either in parallel or in sequence. The situation where you have a lot of nested callbacks, or independent callbacks that have to be synchronized, is often referred to as the “callback hell.”

jQuery helps you avoid all these browser issues but, depending on the version of the library you’re using, it might do it in a slightly different manner. jQuery provides you with two objects, Deferred and Promise, that you can reliably use

in projects Lets c the concept of promises through the use of two objects:
Deferred and **Promise**.

- A promise represents a value that is not yet known
- A deferred represents work that is not yet finished



In JavaScript, a promise is a request to resource from another website, the result of a long calculation either from your server or from a JavaScript function, or the response of a REST service (these are examples of promises), and you perform other tasks while you await the result. When the latter becomes available (the promise is resolved/fulfilled) or the request has failed (the

promise is failed/rejected), you act accordingly. Promises have been widely discussed and such discussions have resulted in two proposals:

- Promises/A
- Promises/A+.

A promise represents the eventual result of an asynchronous operation. The primary way of interacting with a promise is through its then method, which registers callbacks to receive either a promise's eventual value or the reason why the promise cannot be fulfilled.

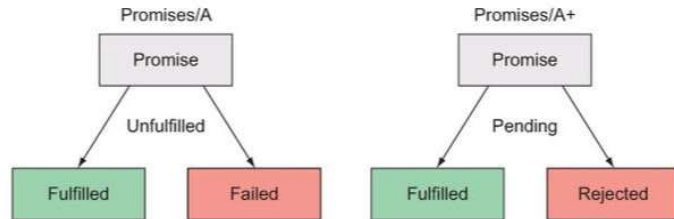
Promises/A+ specifications

The then() method described by the Promises/A+ proposal is the core of promises. The then() method accepts two functions: one to execute in the event that the promise is fulfilled and the other if the promise is rejected. When a promise is in one of these states, regardless of which one, it's settled. If a promise is neither fulfilled nor rejected (for example, if you're still waiting for the response of a server calculation), it's pending.

A promise represents the eventual value returned from the single completion of an operation. A promise may be in one of the three states, unfulfilled, fulfilled, and failed. The promise may only move from unfulfilled to fulfilled, or unfulfilled to failed.

Promises/A specifications

As per below Fig, the terminology for the Promise object's definition is a bit different. The Promises/A proposal defines the unfulfilled, fulfilled, and failed states, whereas the Promises/A+ proposal uses the pending, fulfilled, and rejected states.



These proposals outline the behavior of promises and not the implementation, so libraries implementing promises have a `then()` method in common but may differ for other methods exposed.

The Deferred and Promise objects

The Deferred object was introduced in jQuery 1.5 as a chainable utility used to register multiple callbacks into callback queues, invoke callback queues, and relay the success or failure state of any synchronous or asynchronous function. This object can be used for many asynchronous operations, like Ajax requests and animations, but also with JavaScript timing functions. Together with the Promise object, it represents the jQuery implementation of promises.

The Promise object is created starting from a Deferred object or a jQuery object and possesses a subset of the methods of the Deferred object (`always()`, `done()`, `fail()`, `state()`, and `then()`). Deferred objects are typically used if you write your own function that deals with asynchronous callbacks. So, function is the producer of the value and you want to prevent users from changing the state of the Deferred.

The Deferred methods

In jQuery, a Deferred object is created by calling the `$.Deferred()` constructor. The syntax of this function is as follows.

Method syntax: `$.Deferred`

`$.Deferred([beforeStart])`

A constructor function that returns a chainable utility object with methods to register multiple callbacks into callback queues, invoke callback queues, and relay the success or failure state of any synchronous or asynchronous function. It accepts an optional function to execute before the constructor returns.

Parameters

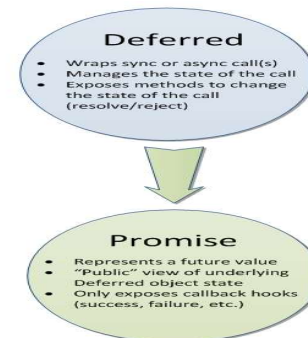
beforeStart (Function) A function that's called before the constructor returns. The function accepts a Deferred object used as the context (`this`) of the function.

Returns

The Deferred object.

Example:

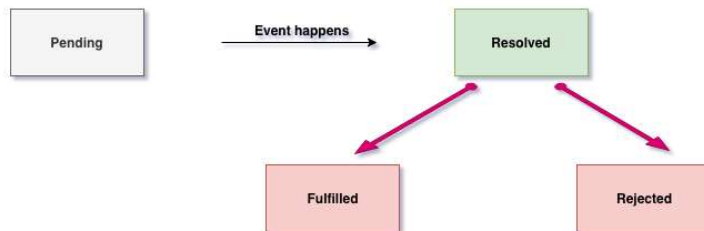
`$.Deferred().html('Promises are great!');`



In jQuery, a promise can be resolved (the promise is successful), rejected (an error occurred), or pending (the promise is neither resolved nor rejected). These states can be reached in two ways. The first is determined by code that you or another developer has written and with an explicit call to methods like `deferred.resolve()`, `deferred.resolveWith()`, `deferred.reject()`, or `deferred.rejectWith()`. These methods, as we'll discuss in a few moments, allow you to either resolve or reject a promise. The second is determined by the success or the failure of a jQuery function—for example, `$.ajax()`—so you don't have to call any of the previously mentioned methods yourself.

A promise is an object that represents the return value or the thrown exception that the function may eventually provide. In other words, a promise represents a value that is not yet known. A promise is an asynchronous value. The core idea behind promises is that a promise represents the result of an asynchronous operation. A Promise has 3 possible states – Pending – Fulfilled – Rejected.

THREE STATES OF A PROMISE



Example: Simple Functional Transform

```

var author = getAuthors();

var authorName = author.name;

// becomes

var authorPromise = getAuthors().then(function (author) {

    return author.name; });
  
```

Method syntax: `promise`

`promise([type][, target])`

Returns a dynamically generated Promise object that's resolved once all actions of a certain type bound to the collection, queued or not, have finished. By default, type is `fx`, which means the returned Promise is resolved when all animations of the selected elements have completed.

Parameters

type (String) The type of queue that has to be observed. The default value is `fx`, which represents the default queue for the effects.

target (Object) The object onto which the promise methods have to be attached.

Returns

A Promise object.

Example:

```

$('p')

    .promise()
  
```

```
.then(function(value) { console.log(value); });
```

Deferred Vs Promise

The main difference between callbacks and promises is that with callbacks you tell the executing function what to do when the asynchronous task completes, whereas with promises the executing function returns a special object to you (the promise) and then you tell the promise what to do when the asynchronous task completes.

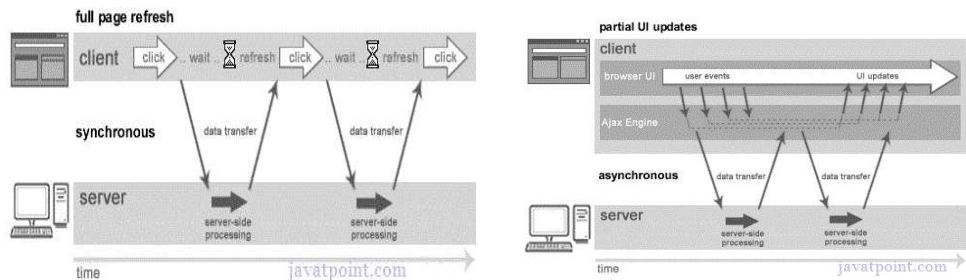
Deferred	Promise
You cannot use \$.Promise();	new \$.Deferred(); OR \$.Deferred()
A deferred object is an object than can create a promise and change its state to resolved or rejected. Deferreds are typically used if you write your own function and want to provide a promise to the calling code. You are the producer of the value.	A promise is, as the name says, a promise about a future value. You can attach callbacks to it to get that value. The promise was "given" to you and you are the receiver of the future value. You cannot modify the state of the promise. Only the code that created the promise can change its state.
You can call use resolve or reject etc.	The Promise exposes only the Deferred methods needed to attach additional handlers or determine the state (then, done, fail, always, pipe, progress, and state), but not ones that change the state (resolve, reject, notify, <u>resolveWith</u> , <u>rejectWith</u> , and <u>notifyWith</u>).

Single Page Applications & Asynchronous Communication

AJAX (Asynchronous JavaScript And XML)

A **synchronous** request blocks the client until operation completes i.e. browser is unresponsive. In such case, javascript engine of the browser is blocked.

An **asynchronous** request doesn't block the client i.e. browser is responsive. At that time, user can perform another operations also. In such case, javascript engine of the browser is not blocked.



Asynchronous Communication

AJAX(Asynchronous JavaScript And XML)

- Traditional web applications, upon request from user like clicking a link or submitting a form, a new page is loaded with requested resources
- Asynchronous applications, upon user action, updates a part of the page without reloading the entire page
- Approaches include
 - Setting src property of iFrame or img element
 - A more elegant and complete approach is use of XHR or XMLHttpRequest object
- First create an XHR object using
`var xhr = new XMLHttpRequest();`



Single Page Applications

Introduction

- Instead of the default method of the browser loading entire new pages, a single-page application (SPA) interacts with the web browser by dynamically **rewriting the current web page with new data** from the web server
- Resources are dynamically loaded and added to the page as necessary, usually in response to user actions
- The page does not reload at any point in the process, nor does it transfer control to another page
- Can be built using
 - AJAX
 - Frameworks like ReactJS, AngularJS



jQuery

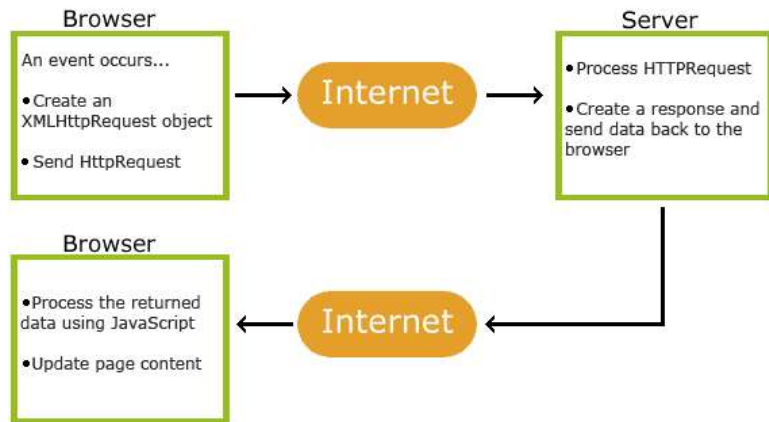
AJAX- XHR

An object of XMLHttpRequest is used for asynchronous communication between client and server.

XMLHttpRequest object plays a important role.

1. User sends a request from the UI and a javascript call goes to XMLHttpRequest object.
2. HTTP Request is sent to the server by XMLHttpRequest object.
3. Server interacts with the database using JSP, PHP, Servlet, ASP.net etc.
4. Data is retrieved.
5. Server sends XML data or JSON data to the XMLHttpRequest callback function.
6. HTML and CSS data is displayed on the browser.





Properties / Methods	Description
<code>open(method, url [, asynchronous])</code>	Initializes the request in preparation for sending to the server. The <i>method</i> parameter is the HTTP method to use, for example "GET" or "POST". The <i>url</i> is the relative or absolute URL the request will be sent to. The optional <i>asynchronous</i> parameter indicates whether <i>send()</i> returns immediately or after the request is complete (default is true, meaning it does not wait for response to come back)
<code>onreadystatechange</code>	Function to call whenever the readyState changes
<code>send([body])</code>	Initiates the request to the server. The <i>body</i> parameter should contain the body of the request, i.e., a string containing <i>fieldname=value&fieldname2=value2...</i> for POSTs or a null value for GET request

<code>readyState</code>	Integer indicating the state of the request, either: 0 (uninitialized) 1 (loading) 2 (response headers received) 3 (some response body received) 4 (request complete)
<code>status</code>	HTTP status code returned by the server (e.g., 200, 404, etc.) 200: "OK" 403: "Forbidden" 404: "Page not found"
<code>responseText</code>	Full response from the server as a string (<code>responseType</code> property is set to "text" – default)
<code>responseXML</code>	A Document object representing the server's response parsed as an XML document (<code>responseType</code> property is set to "document")
<code>response</code>	Any other type of response received ((<code>responseType</code> property is set to "blob" or "json")

```

let xmlhttp = new XMLHttpRequest();
xmlhttp.open("GET", filepath, true);
xmlhttp.onreadystatechange=handler;
[xmlhttp.responseType="json"|"document"|"blob"] // default text
xmlhttp.send(null);

function handler() {
    if(this.readyState == 4 && this.status == 200) {
        // use this.response (json/blob) or this.responseText (text) or
        // this.responseXML (document) to update a part of the page
    }
}
  
```

Unit 2: HTML5, JQuery and Ajax

Asynchronous Communication- XHR

AJAX is not a programming language, but a technique that incorporates a client-side script (i.e. a script that runs in a user's browser) that communicates with a web server. Further, its name is somewhat misleading: while an AJAX application might use XML to send data, it could also use just plain text or JSON text. But generally, it uses an XMLHttpRequest object in your browser to request data from the server and JavaScript to display the data.

XMLHttpRequest supports both synchronous and asynchronous communications. In general, however, asynchronous requests should be preferred to synchronous requests for performance reasons.

Synchronous requests block the execution of code which causes "freezing" on the screen and an unresponsive user experience.

Fetch API

The Fetch API provides an interface for fetching resources. It will seem familiar to anyone who has used XMLHttpRequest, but this API provides a more powerful and flexible feature set.

AJAX can access the server both synchronously and asynchronously:

- **Synchronously**, in which the script stops and waits for the server to send back a reply before continuing.
- **Asynchronously**, in which the script allows the page to continue to be processed and handles the reply if and when it arrives.

Components of AJAX

The AJAX cannot work independently. It is used in combination with other technologies to create interactive Web pages that are described in the following list:

JavaScript:

- Loosely typed scripting language.
- JavaScript function is called when an event occurs in a page.
- Glue for the whole AJAX operation.

DOM:

- API for accessing and manipulating structured documents.
- Represents the structure of XML and HTML documents.

CSS:

- Allows for a clear separation of the presentation style from the content and may be changed programmatically by JavaScript.

XMLHttpRequest:

- JavaScript object that performs asynchronous interaction with the server.

XMLHttpRequest Methods

- **abort()**

Cancels the current request.

- **getAllResponseHeaders()**

Returns the complete set of HTTP headers as a string.

- **getResponseHeader(headerName)**

Returns the value of the specified HTTP header.

- **open(method, URL)**
- **open(method, URL, async)**
- **open(method, URL, async, userName)**
- **open(method, URL, async, userName, password)**

Specifies the method, URL, and other optional attributes of a request.

The method parameter can have a value of "GET", "POST", or "HEAD". Other HTTP methods such as "PUT" and "DELETE" (primarily used in REST applications) may be possible.

The "async" parameter specifies whether the request should be handled asynchronously or not. "true" means that the script processing carries on after the send() method without waiting for a response, and "false" means that the script waits for a response before continuing script processing.

- **send(content)**

Sends the request.

- **setRequestHeader(label, value)**

Adds a label/value pair to the HTTP header to be sent.

XMLHttpRequest Properties

- **onreadystatechange**

An event handler for an event that fires at every state change.

- **readyState**

The readyState property defines the current state of the XMLHttpRequest object.

The following table provides a list of the possible values for the readyState property –

State	Description
0	The request is not initialized.
1	The request has been set up.
2	The request has been sent.
3	The request is in process.
4	The request is completed.

readyState = 0 After you have created the XMLHttpRequest object, but before you have called the open() method.

readyState = 1 After you have called the open() method, but before you have called send().

readyState = 2 After you have called send().

readyState = 3 After the browser has established a communication with the server, but before the server has completed the response.

readyState = 4 After the request has been completed, and the response data has been completely received from the server.

- **responseText**

Returns the response as a string.

- **responseXML**

Returns the response as XML. This property returns an XML document object, which can be examined and parsed using the W3C DOM node tree methods and properties.

- **status**

Returns the status as a number (e.g., 404 for "Not Found" and 200 for "OK").

- **statusText**

Returns the status as a string (e.g., "Not Found" or "OK").

How to choose between GET & POST?

Purpose of GET - to GET information , intended to be used when you are reading information to display on the page. Browsers will automatically cache the result from a GET request and if the same GET request is made again then they will display the cached result rather than rerunning the entire request. A GET call is retrieving data to display in the page and data is not expected to be changed on the server by such a call and so re-requesting the same data should be expected to obtain the same result.

POST method is intended to be used where you are updating information on the server . Results returned from server . A POST call will therefore always obtain the response from the server rather than keeping a cached copy of the prior response. If the value to be retrieved is expected to vary over time as a result of other processes updating it then add a current time parameter to what you are passing in your GET call. These criteria is not only for GET and POST for your Ajax calls but also to GET or POST when processing forms on your web page as well.



WEB TECHNOLOGIES

XML vs. JSON

Vinay Joshi

Compiled by **Dr.Sarasvathi V**

Associate Professor

Department of Computer Science and Engineering

XML vs. JSON

XML

- XML stands for “extensible markup language”.
- XML was designed in 1996 and officially became a W3C standard in 1998.
- It was created to better represent data formats with a hierarchical structure.
- HTML
 - Display data and focus on how the data looks.
- XML
 - Structures, stores, and transfers information and describes what the data is.



XML Vs JSON HTML5

XML PROS

- This data format fully supports hierarchical data structures and is very appropriate when receiving complex data as a response.
- It is also very human readable. Most browsers have built in XML readers that allow you to inspect XML files.
- Since XML was the first standard hierarchical data format, most APIs have built in functionality to automatically convert XML data streams into native data structures like objects.



XML Vs JSON HTML5

XML CONS

- This data format is about three times as large as CSV. This is because each data element has an associated open and close parameter tag.



XML vs. JSON

JSON

- JSON stands for (Javascript Object Notation).
- It was invented in 2001 and became popularized by Yahoo and Google in 2005 and 2006.
- It was created as an alternative to XML. Like XML, however, it represents hierarchical data with the use of commas, curly braces and brackets.

XML Vs JSON HTML5

JSON CONS

- This data format has a little bit less support than XML.
- Since JSON is relatively newer than XML, fewer APIs exist to automatically convert JSON to native data structures.
- However, this is rapidly changing because newer APIs and plugins are supporting both XML and JSON.



XML Vs JSON HTML5

JSON PROS

- This data format supports hierarchical data while being smaller in size than XML.
- As its name implies, it was also created to more easily **parse data into native Javascript** objects, making it very useful for web applications.
- JSON is the best of both worlds with respect to CSV and XML. It's simple and compact like CSV, but supports hierarchical data like XML. Unlike XML, JSON formats are only about twice as large as CSV formats.



XML vs. JSON

Format

XML

```
<empinfo>
  <employees>
    <employee>
      <name>James Kirk</name>
      <age>40</age>
    </employee>
    <employee>
      <name>Jean-Luc Picard</name>
      <age>45</age>
    </employee>
    <employee>
      <name>Wesley Crusher</name>
      <age>27</age>
    </employee>
  </employees>
</empinfo>
```

JSON

```
{ "empinfo" :
  {
    "employees" : [
      {
        "name" : "James Kirk",
        "age" : 40,
      },
      {
        "name" : "Jean-Luc Picard",
        "age" : 45,
      },
      {
        "name" : "Wesley Crusher",
        "age" : 27,
      }
    ]
  }
}
```

XML vs. JSON

Differences

JSON	XML
Based on JavaScript language	Derived from SGML
Way of representing objects	Markup language to represent data items
No support for namespaces	Supports namespaces
Support arrays	Doesn't support arrays
Files are very easy to read as compared to XML	Documents are comparatively difficult to read and interpret
Doesn't use end tag	Has start and end tags
Less secure	More secured than JSON
Doesn't supports comments	Supports comments
Supports only UTF-8 encoding	Supports various encoding

XML Vs JSON HTML5

JSON

- JavaScript has a built in function for converting JSON strings into JavaScript objects:
`JSON.parse()`
- JavaScript also has a built in function for converting an object into a JSON string:
`JSON.stringify()`
- You can receive pure text from a server and use it as a JavaScript object.
- You can send a JavaScript object to a server in pure text format.
- You can work with data as JavaScript objects, with no complicated parsing and translations.



XML Vs JSON HTML5

JSON

JSON String

```
'{"name": "John", "age": 30, "car": null}'
```



XML Vs JSON HTML5

XML Parser

A text string is defined:

```
text = "<bookstore><book>" +  
      "<title>Everyday Italian</title>" +  
      "<author>Giada De Laurentiis</author>" +  
      "<year>2005</year>" +  
      "</book></bookstore>";
```



XML Vs JSON HTML5

XML Parser

Parses a text string into an XML DOM object, and extracts the info from it with JavaScript:

An XML DOM parser is created:

```
parser = new DOMParser();
```

The parser creates a new XML DOM object using the text string:

```
xmlDoc = parser.parseFromString(text, "text/xml");
```



XML Vs JSON HTML5

XML

XML DOM object to XML string:

XMLSerializer interface provides `serializeToString()` method to construct an XML string

```
var oSerializer = new XMLSerializer();  
var sXML = oSerializer.serializeToString(xmlDOM);  
console.log(sXML);
```



THANK YOU

Vinay Joshi

Compiled by **Dr.Sarasvathi V**

Department of Computer Science and Engineering

Unit 2: HTML5, JQuery and Ajax

XML Vs JSON

XML Vs JSON

JSON, XML, are Text-file formats that can be used to store structured data that can be handy for embedded and Web applications.

XML (Extensible Markup Language) has been around for more than 3 decades now and it is an integral part of every web application. Be it a configuration file, mapping document or a schema definition, XML made life easier for data interchange by giving a clear structure to data and helping in dynamic configuration and loading of variables!

JSON stores all of its data in a map format (key/value pairs) that was neat and easier to comprehend. JSON is said to be slowly replacing XML because of several benefits like ease of data modeling or mapping directly to domain objects, more predictability and easy to understand the structure. JSON is just a data format whereas XML is a markup language.

Structure of XML vs JSON :

XML (Extensible Markup Language)	JSON (JavaScript Object Notation)
<pre> 01 Adam Cloud computing </technology > Development </pre>	<pre> { "employees": [{ "id": "01", "name": "Adam", "technology": "Cloud computing", "title": "Engineer", "team": "Development" }] } </pre>

Difference between XML and JSON:

XML (Extensible Markup Language)	JSON (JavaScript Object Notation)
XML is a markup language, not a programming language, that has tags to define elements.	JSON is just a format written in JavaScript.
XML data is stored as a tree structure. Example –	Data is stored like a map with key value pairs. Example –
<pre> 2001 Varsha 2002 Akash </pre>	<pre> {"employees": [{"id": "2001", "name": "Varsha"}, {"id": "2002", "name": "Akash"}]} </pre>

Web Technologies

Can perform processing and formatting documents and objects.	It does not do any processing or computation
Bulky and slow in parsing, leading to slower data transmission	Very fast as the size of file is considerably small, faster parsing by the JavaScript engine and hence faster transfer of data
Supports namespaces, comments and metadata	There is no provision for namespace, adding comments or writing metadata
Document size is bulky and with big files, the tag structure makes it huge and complex to read.	Compact and easy to read, no redundant or empty tags or data, making the file look simple.
Doesn't support array directly. To be able to use array, one has to add tags for each item.	Supports array which can be accessed as –
science maths computers	<pre>x = student.subjects[i];</pre> where "subjects" is an array as – <pre>"subjects": ["science", "math", "computers"]</pre>
Supports many complex data types including charts, images and other non-primitive data types.	JSON supports only strings, numbers, arrays Boolean and object. Even object can only contain primitive types.
XML supports UTF-8 and UTF-16 encodings.	JSON supports UTF as well as ASCII encodings.
XML structures are prone to some attacks as external entity expansion and DTD validation are enabled by	JSON parsing is safe almost all the time except if JSONP is used, which can lead to Cross-Site Request

Web Technologies

default. When these are disabled, XML parsers are safer.	Forgery (CSRF) attack.
Though the X is AJAX stands for XML, because of the tags in XML, a lot of bandwidth is unnecessarily consumed, making AJAX requests slow.	As data is serially processed in JSON, using it with AJAX ensures faster processing and hence preferable. Data can be easily manipulated using eval() method.



WEB TECHNOLOGIES

jQuery AJAX and fetch() methods

Vinay Joshi

Department of
Computer Science and Engineering

jQuery jQuery Ajax Methods

The **ajax()** method is used to perform an AJAX (asynchronous HTTP) request.

All jQuery AJAX methods use the **ajax()** method. This method is mostly used for requests where the other methods cannot be used.

Syntax

`$.ajax({name:value, name:value, ... })`

The parameters specifies one or more name/value pairs for the AJAX request.

jQuery AJAX methods

Introduction

- jQuery provides methods that use XMLHttpRequest internally to make AJAX requests
- The methods are
 - \$.ajax
 - \$.get
 - \$.post
 - \$.load
- In a single method call, the entire functionality of making an AJAX call using XMLHttpRequest and updating the page can be achieved



jQuery AJAX methods

\$.ajax

- Performs an async AJAX request
- Syntax
 - `$.ajax({name:value, name:value, ... })`
- Example
 - ```
$.ajax({url: "demo_test.txt", method: 'get', success: function(result){
 $("#div1").html(result);
}});
```
  - ```
$.ajax('/jquery/submitData', {
    type: 'POST', // http method
    data: { myData: 'This is my data.' }, // data to submit
    success: function (data, status, xhr) { // success callback function
        $('p').append('status: ' + status + ', data: ' + data);
    },
    error: function (jqXHR, textStatus, errorMessage) {
        $('p').append('Error' + errorMessage);
    }
});
```



jQuery

jQuery Ajax Methods

The **\$.get()** method requests data from the server with an HTTP GET request.

Syntax:

```
$.get(URL,callback);
```

The required URL parameter specifies the URL you wish to request.

The optional callback parameter is the name of a function to be executed if the request succeeds.



jQuery AJAX methods

\$.get

- Syntax
 - \$.get(url, [data],[callback]);
- Example
 - \$.get('/jquery/getjsondata', // url
 - {name:'Steve'}, // request parameters
 - function (data, textStatus, jqXHR) { // success callback function
 - \$('.p').append(data.firstName);
 - };
- Other Variants
 - \$.getJSON(url, [data],[callback]);
 - \$.getScript(url, [data],[callback]);



jQuery AJAX methods

\$.post

- Syntax
 - \$.post(url,[data],[callback],[type]);
- Example
 - \$.post('/jquery/submitJSONData', // url
 - { myData: 'This is my data.' }, // data to be submitted
 - function(data, status, jqXHR) { // success callback function
 - \$('.p').append('status: ' + status + ', data: ' + data);
 - },
 - "json" //response type
 -);
- Internally uses \$.ajax with method="post"



jQuery AJAX methods

\$.load

- Allows HTML or text content to be loaded from a server and added into an existing DOM element
- Syntax
 - \$('selector').load(url,[data],[callback],[type]);
- Example
 - \$('#msgDiv').load('getData', // url
 - { name: 'bill' }, // data
 - function(data, status, jqXHR) { // success callback function
 - console.log('data loaded')
 - };



fetch() AJAX method

Introduction

- Starts the process of fetching a resource from the network
- Returns a promise which is fulfilled once the response is available
- The promise resolves to the Response object representing the response to your request
- The promise does not reject on HTTP errors — it only rejects on network errors. You must use then handlers to check for HTTP errors.
- Syntax
 `const fetchResponsePromise = fetch(resource [, init])`



fetch() AJAX method

Code Example

```
const mydiv = document.querySelector('.my-div');

fetch('resp.html') // returns a promise that resolves on response object
.then(function(response) {
    return response.text(); // returns a promise that resolves on text response
})
.then(function(text) {
    mydiv.innerHTML= text;
});
```



Unit 2: HTML5, JQuery and Ajax

AJAX load() Method

- The load() method loads data from a server and puts the returned data into the selected element.

Syntax: \$(selector).load(URL, data, callback);

- The required URL parameter specifies the URL you wish to load.
- The optional data parameter specifies a set of query string key/value pairs to send along with the request.
- The optional callback parameter is the name of a function to be executed after the load() method is completed. The following example loads the content of the file "test.txt" into a specific <div> element:

```
$("#div1").load("test.txt");
```

It is also possible to add a jQuery selector to the URL parameter. The following example loads the content of the element with id="p1", inside the file "test.txt", into a specific <div> element:

```
$("#div1").load("test.txt #p1");
```

The optional callback parameter specifies a callback function to run when the load() method is completed. The callback function can have different parameters:

- responseTxt - contains the resulting content if the call succeed
- statusTXT - contains the status of the call
- xhr - contains the XMLHttpRequest object

The following example displays an alert box after the load() method completes. If the load() method has succeed, it displays "External content loaded successfully!", and if it fails it displays an error message:

```
$("#button").click(function(){
$("#div1").load("demo_test.txt",function(responseTxt,statusTxt,xhr){
if(statusTxt=="success

alert("External content loaded successfully!");

if(statusTxt=="error")

alert("Error: "+xhr.status+": "+xhr.statusText); });

});
```

HTTP Request: GET vs. POST

Two commonly used methods for a request-response between a client and server.

- GET - Requests data from a specified resource
- POST - Submits data to be processed to a specified resource

• \$.get(URL,callback);

- The required URL parameter specifies the URL you wish to request.
- The optional callback parameter is the name of a function to be executed if the request succeeds.

• \$.post(URL,data,callback);

- The required URL parameter specifies the URL you wish to request.

- The optional data parameter specifies some data to send along with the request.
- The optional callback parameter is the name of a function to be executed if the request succeeds.

Example

```

$("button").click(function(){
$.post("test_post.jsp",
{
name:"Bill Gates",
city:"Seattle"
},
function(data,status)
{
alert("Data: " + data + "\nStatus: " + status);
});
});

```

- The first parameter of \$.post() is the URL we wish to request ("test_post.jsp").
- Then we pass in some data to send along with the request (name and city).

- The JSP script in "test_post.jsp" reads the parameters, process them, and return a result.
- The third parameter is a callback function. The first callback parameter holds the content of the page requested, and the second callback parameter holds the status of the request.

The \$.ajax(settings) or \$.ajax(url, settings)

Used for sending an Ajax request. The settings is an object of key-value pairs. The frequently-used keys are:

- *url*: The request URL, which can be placed outside the *settings* in the latter form.
- *type*: GET or POST.
- *data*: Request parameters (name=value pairs). Can be expressed as an object (e.g., {name:"peter", msg:"hello"}), or query string (e.g., "name=peter&msg=hello").
- *dataType*: Expected response data type, such as text, xml, json, script or html.
- *headers*: an object for request header key-value pairs. The header X-Requested-With:XMLHttpRequest is always added.

Ajax request, by default, is asynchronous. In other words, once the .ajax() is issued, the script will not wait for the response, but continue into the next statement, so as not to lock up and freeze the screen.

NOTE: \$ is a shorthand (alias) for the jQuery object. \$() is an alias for jQuery() function for Selector. \$.ajax() is a global function (similar to class method in an OO language).

The Fetch API

The Fetch API provides a fetch() method defined on the window object, which you can use to perform requests. This method returns a Promise that can be used to retrieve the response of the request.

The fetch method only has one mandatory argument, which is the URL of the resource you wish to fetch. The Fetch API is a modern interface that allows you to make HTTP requests in the web browsers. The fetch() method is available in the global scope that instructs the browser to send a request to a provided URL.

Sending a Request

The fetch() has only one parameter which most of the time is the URL of the resource that you want to fetch:

```
let response = fetch(url);
```

The fetch() method returns a Promise so you can use the then() and catch() methods to handle it:

```
fetch(url)
```

```
.then(response => {  
  
  // handle the response  
  
})  
  
.catch(error => {  
  
  // handle the error  
  
});
```

When the request completes, the resource is available. At this time, the promise will resolve into a Response object.

The Response object is the API wrapper for the fetched resource. The Response object has a number of useful properties and methods to inspect the response.

Reading a Response

If the contents of the response are in the raw text format, you can use the text() method. The text() method returns a Promise that resolves with the complete contents of the fetched resource:

```
fetch('/readme.txt')
```

```
.then(response => response.text())  
  
.then(data => console.log(data));
```

Or async/await can be used:

```
async function fetchText() {  
  
  let response = await fetch('/readme.txt');  
  
  let data = await response.text();  
  
  console.log(data);  
  
}
```

Besides the text() method, the Response object has other methods such as json(), blob(), formData() and arrayBuffer() to handle respective data.

Handling status codes of a response

The Response object provides the status code and status text via the status and statusText properties. When a request is successful, the status code is 200 and status text is OK:

```
async function fetchText() {  
  
  let response = await fetch('/readme.txt');
```

```
console.log(response.status); // 200

console.log(response.statusText); // OK

if (response.status === 200) {

    let data = await response.text();

    // handle data

}

}
```

fetchText();

Output:

200

OK

If the requested resource doesn't exist, the response code is 404:

```
let response = await fetch('/non-existence.txt');

console.log(response.status); // 400

console.log(response.statusText); // OK
```

Output:

400

Not Found

If the requested URL throws a server error, the response code will be 500.

If the requested URL is redirected to the new one with the response 300-309, the status of the Response object is set to 200. In addition the redirected property is set to true.

The fetch() returns a promise that rejects when a real failure occurs such as a web browser timeout, a loss of network connection, and a CORS violation.

WEB TECHNOLOGIES

Web Development Stack

Prof. Vinay Joshi and Dr.Sarasvathi V
Department of Computer Science and Engineering

Web Development Stack

Introduction

- A set of tools typically used in tandem to develop web apps.
- Refers to the technologies that that individual developer specializes in and use together to develop new pieces of software.
 - Front End web development stack
 - Back End web development stack
 - Full web development stack
 - Web technology sets that include all the essential parts of a modern app: the **frontend framework**, the **backend solution** and the **database (relational or document-oriented)**



Web Development Stack

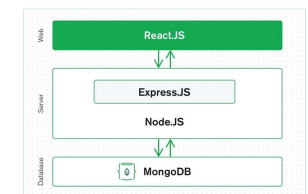
Full web development Stack

Web Development Stack

MERN - Introduction

- As the web development world is continually changing, its technology stacks too changing
- Top web development stacks
 - MEAN
 - MERN
 - Meteor.js
 - Flutter
 - The serverless Technology stack
 - The LAMP technology stack
 - Ruby on Rails Tech Stack

- Stands for **MongoDB, Express, React, Node**
 - MongoDB - document database
 - Express(.js) - Node.js web framework
 - React(.js) - a client-side JavaScript library
 - Node(.js) - the premier JavaScript web server, runtime environment



- Allows you to easily construct a **3-tier architecture** (frontend, backend, database) entirely using JavaScript and JSON MERN

Web Development Stack

React.JS

- Front End Library (The top tier of MERN Stack)
- The declarative JavaScript Library for creating dynamic client-side applications
- Build up complex interfaces through
 - simple Components,
 - connect them to data on your backend server, and
 - render them as HTML
- Strengths:
 - Handling stateful, data-driven interfaces
 - Great support for forms, error handling, events, lists, etc.



Web Development Stack

MongoDB

- Database Server (The bottom tier of MERN Stack)
- Application data store(user profiles, content, comments, uploads, events, etc.)
- JSON documents created in the front end can be stored directly in MongoDB for later retrieval through Node.JS and Express.JS



Web Development Stack

Node.JS and Express.JS

- Server side Framework (The middle tier of MERN Stack)
- Express.js server-side framework, running inside a Node.js server
- Has powerful models for URL routing (matching an incoming URL with a server function), and handling HTTP requests and responses
- In turn use Node.js MongoDB drivers, either via callbacks for using Promises, to access and update data in your MongoDB database



React.JS

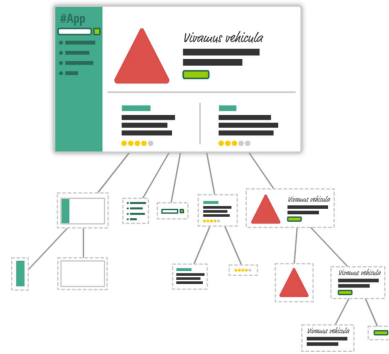
Introduction

- An open-source **JavaScript library that is used for building user interfaces** specifically for **single-page applications**.
- Used for handling the **view layer for web and mobile apps**.
- Allows developers to create **large web applications that can change data**, without reloading the page.
- Also allows to **create reusable UI components**.
- The main purpose of React is to **be fast, scalable, and simple**.
- **Works only on user interfaces** in the application.
- First created **by Jordan Walke**, a software engineer working for Facebook. First deployed on Facebook's newsfeed in 2011 and on Instagram.com in 2012.



React.JS

Introduction



React.JS

How to create a React App?

• Two ways:

- **Using node package manager(npm):** To setup a build environment for React that typically involved use of npm (node package manager), webpack, and Babel
- **Directly importing Reactjs library in HTML Code.** Defined in two .js files (**React** and **ReactDOM**)
 - `<script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>`
 - `<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>`
 - The files differ for development and production.
 - When deploying, replace "development.js" with "production.min.js"



React.JS

Why React?

• Properties

- Declarative, Simple, Component based, Supports server side, Mobile support, Extensive, Fast Easy to learn

• Single way data flow

- A set of immutable values are passed to the components renderer as properties in its HTML tags. The component cannot directly modify any properties but can pass a call back function with the help of which we can do modifications.
- This complete process is known as "**properties flow down; actions flow up**".

• Virtual DOM

- Creates an in-memory data structure cache which computes the changes made and then updates the browser.
- Allows a special feature that enables the programmer to code as if the whole page is rendered on each change whereas react library only renders components that actually change



React.JS

Creating React App Using node package manager(npm)

To create a new React app, you can use the **create-react-app** tool. Here's how you can do it step by step:

1. Download Node.js

Before you start, make sure Node.js is installed on your machine. Download and install it from [Node.js](https://nodejs.org/).

2. Install **create-react-app** (if you haven't already)

You need to globally install **create-react-app** using npm:

npm install -g create-react-app





3. Create a New React App

Once the `create-react-app` tool is installed, you can create a new React project by running:

```
npx create-react-app my-app-1
```

Replace `my-app-1` with your desired project name. This command will generate a new folder with all the necessary files and dependencies for your React project.



4. Navigate to the Project Directory

After the app is created, navigate into the project folder:

```
cd my-app-1
```



5. Start the Development Server

To start your React app, run the following command:

```
npm start
```

This will start the development server, and you should be able to see your React app running at `http://localhost:3000` in your web browser.

6. Open the Project in Your Code Editor

Open the project folder (`my-app-1`) in your preferred code editor (like VS Code) to start building your app.



You can check the installed version of React using the terminal by running the following commands:

1. Check React Version

In your project directory, run the following command to check the version of `react`:

```
npm ls react
```

Checking Installed Versions of React and React DOM

2. Check React DOM Version

To check the version of `react-dom`, run:

`npm ls react-dom`

These commands will display the installed version of `react` and `react-dom`.

React Component Example codes

• Stateless Functional Component

```
function Demo(props) {
  return <h1> Welcome to REACT JS, {props.Name} </h1>;
}
```

• Stateful Class Component

```
class HelloClass extends React.Component
{
  render()
  {
    return React.createElement('h1', null, 'welcome to REACT JS');
  }
}
```

With props

```
class Demo extends React.Component{
  render(){
    return <h1> Welcome to REACT JS, {this.props.Name} </h1>;
  }
}
```

React Component

- A user control that has **code to represent visual interfaces and data**
- An isolated piece of code which can be reused in one or the other module
- Contains a root component in which other subcomponents are included
- 2 types of components in React.js
 - **Stateless Functional Component**
 - Includes simple JavaScript functions and immutable properties, i.e., the value for properties cannot be changed.
 - Use hooks to achieve functionality for making changes in properties using JS.
 - Used mainly for UI.
 - **Stateful Class Component**
 - Classes which extend the Component class from React library. The class component must include the render method which returns HTML.



React Component – Few key terms

- **Placeholder:** A place where the component will load
- **Create component:** Stateless or stateful
- **Call Component:** `ReactDOM.render()` is responsible for rendering a React component.
 - The first parameter is the component class name. Second parameter is the destination where the component is to be rendered. In our case, we render component inside the div `id='root'`.
- Example: `ReactDOM.render(

React.createElement(HelloClass, null, null),
 document.getElementById('root')

);`



- **React 17**

```
ReactDOM.render(
  <h1>Batman</h1>,
  document.querySelector("#container")
);
```

The render method takes two arguments:

The HTML-like elements (aka JSX) you wish to output

The location in the DOM that React will render the JSX into

- **React 18**

Instead of ReactDOM.render, createRoot is used

```
const root = createRoot(container);
root.render(element);
```

Example:

```
var destination = document.querySelector("#container");
const root = ReactDOM.createRoot(destination);
root.render(<h1>React World</h1>);
```

Create a React root for the supplied container and return the root.

The root can be used to render a React element into the DOM with render

- Uses a special syntax which allows you to **mix HTML with JavaScript**
- JSX is a JavaScript XML used in React applications. An extension to JavaScript.
- Uses HTML syntax to create elements and components.
- Has tag name, attributes, and children.
- JSX compiles the code into pure JavaScript which can be understood by the browser.
- Include the library: `<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>`

```
<script type="text/babel">
  ReactDOM.render(<h1>Welcome to REACTJS</h1>,
    document.getElementById('root')
  );
```



```
<!DOCTYPE html>
<html>
<head>
  <title>React 18 App</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<h1>Welcome to REACTJS</h1>);
  </script>
</body>
</html>
```

- A JavaScript compiler that can translate markup or programming languages into JavaScript.
- Available for different conversions.
- React uses Babel to convert JSX into JavaScript.
- Please note that `<script type="text/babel">` is needed for using Babel

```
<script type="text/babel">
  ReactDOM.render(<h1>Welcome to REACTJS</h1>,
    document.getElementById('root')
  );
```



React.JS

What is Babel?



React.JS

Visuals Defined using JSX vs Pure JavaScript



- A JavaScript compiler that can translate markup or programming languages into JavaScript.
- Available for different conversions.
- React uses Babel to convert JSX into JavaScript.

```
import React from 'react';  
import ReactDOM from 'react-dom/client';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<h1>Welcome to REACTJS</h1>);
```

JSX

```
ReactDOM.render(  
  <div>  
    <h1>Batman</h1>  
    <h1>Iron Man</h1>  
    <h1>Nicolas Cage</h1>  
    <h1>Mega Man</h1>  
  </div>,  
  destination );
```

JavaScript

```
ReactDOM.render(  
  React.createElement ( "div", null,  
    React.createElement ( "h1", null, "Batman" ),  
    React.createElement ( "h1", null, "Iron Man" ),  
    React.createElement ( "h1", null, "Nicolas Cage" ),  
    React.createElement ( "h1", null, "Mega Man" ),  
    destination);
```



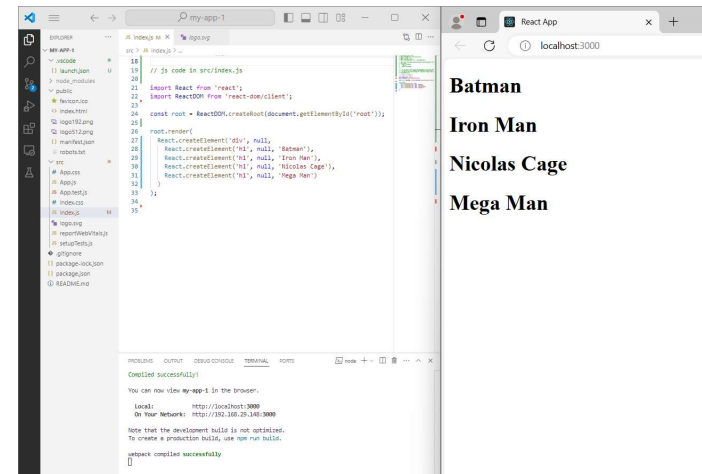
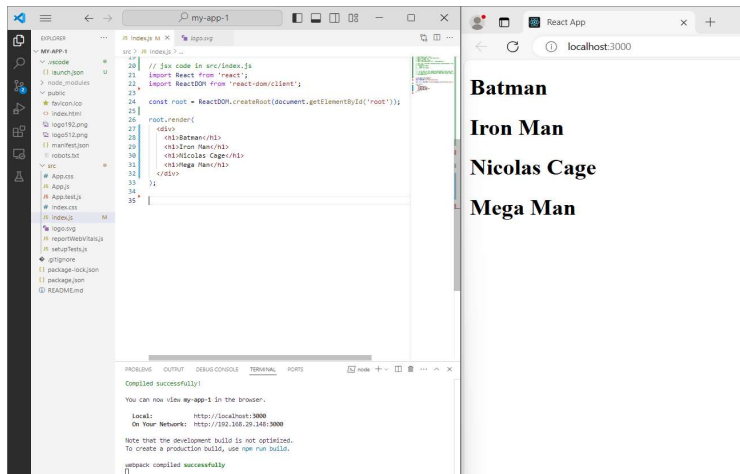
React.JS

Visuals Defined using JSX vs Pure JavaScript



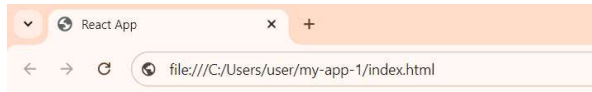
React.JS

Visuals Defined using JSX vs Pure JavaScript



React.JS Demonstration

- Using React, print welcome to REACTJS on the web page
- Use JSX and NonJSX both



Welcome to REACTJS

React.JS Demonstration

Non-JSX Approach

Create index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>React App</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    function App() {
      return React.createElement('h1', null, 'Welcome to REACTJS');
    }

    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(React.createElement(App));
  </script>
</body>
</html>
```



React.JS Demonstration

Directly in HTML with React

1. JSX Approach

Create index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>React App</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
  <div id="root"></div>
  <script type="text/babel">
    function App() {
      return <h1>Welcome to REACTJS</h1>;
    }

    const root = ReactDOM.createRoot(document.getElementById('root'));
    root.render(<App />);
  </script>
</body>
</html>
```



THANK YOU

Vinay Joshi and Dr.Sarasvathi V
Department of Computer Science and Engineering

vinayj@pes.edu

+91 8026726622

sarsvathiv@pes.edu



Unit 2: ReactJS

MERN and REACT.js

Introduction to Web Development

Web development stack is nothing but a set of tools typically used in tandem to develop web apps. It refers to the technologies that individual developer specializes in and use together to develop new pieces of software.

Web technology sets that include all the essential parts of a modern app are as follows: the **frontend framework**, the **backend solution** and the **database (relational or document-oriented)**



Introduction to stack:

Any web application made by using multiple technologies. The combination of these technologies is called a “**stack**,” popularized by the LAMP stack, which is an acronym for Linux, Apache, MySQL, and PHP, which are all open-source components. As the web development world is continually changing, its technology stacks too changing. Top web development stacks are

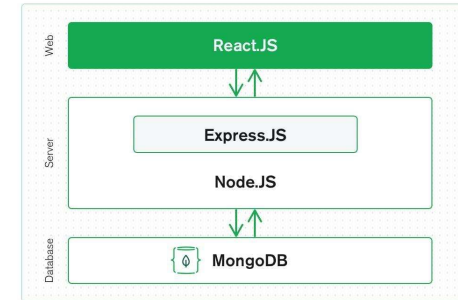
- MEAN
- MERN
- Meteor.js
- Flutter
- The serverless Technology stack
- The LAMP technology stack
- Ruby on Rails Tech Stack

Unit 2: ReactJS

MERN stands for **MongoDB, Express, React, Node**

- MongoDB - Document database
- Express(.js) - Node.js web framework
- React(.js) - A client-side JavaScript library
- Node(.js) - The premier JavaScript web server

Allows you to easily construct a 3-tier architecture (frontend, backend, database) entirely using JavaScript and JSON.



Why MERN?

Ideally suited for web applications that have a large amount of interactivity built into the front-end.

- JavaScript Everywhere
- JSON Everywhere
- Isomorphic

REACT.JS

Free and open source front end JS Library for building UI and UI Components. Maintained by Facebook, a community of individual developers and companies. This can be used as base in the development of Single page applications and mobile applications and

Unit 2: ReactJS

is concerned with state management and rendering the state to DOM.

It is the declarative JavaScript Library for creating dynamic client-side applications

Builds up complex interfaces through simple Components, connect them to data on your backend server, and render them as HTML. Provides support for forms, error handling, events and render them as HTML.

Key points about React.js

- **Properties of React:**
Declarative, Simple, Component based, Supports server side, Mobile support, Extensive, Fast, Easy to learn
- **Single way data flow**
 - A set of immutable values are passed to the components render as properties in its HTML tags. The component cannot directly modify any properties but can pass a call back function with the help of which we can do modifications.
 - This complete process is known as “**properties flow down; actions flow up**”.
- **Virtual DOM**
 - Creates an in-memory data structure cache which computes the changes made and then updates the browser.
 - Allows a special feature that enables the programmer to code as if the whole page is rendered on each change whereas react library only renders components that actually change

Creation of React App

Can be done in two ways

- **Using node package manager(npm):** To setup a build environment for React that typically involved use of npm (node package manager), webpack, and Babel

Unit 2: ReactJS

- **Directly importing Reactjs library in HTML Code.** Defined in two .js files (**React** and **ReactDOM**)
 - `<script src="https://unpkg.com/react@17/umd/react.development.js" crossorigin></script>`
 - `<script src="https://unpkg.com/react-dom@17/umd/react-dom.development.js" crossorigin></script>`
 - The files differ for development and production.
When deploying, replace "development.js" with "production.min.js"

React is used for handling the **view layer for web and mobile apps**. Allows developers to create **large web applications that can change data**, without reloading the page. Also allows to **create reusable UI components**. The main purpose of React is to **be fast, scalable, and simple. Works only on user interfaces** in the application

React Elements:

The browser DOM is made up of DOM elements. Similarly, the React DOM is made up of React elements. DOM elements and React elements may look the same, but they are actually quite different. A React element is a description of what the actual DOM element should look like. In other words, React elements are the instructions for how the browser DOM should be created. Syntax: `React.createElement(type, {props}, children);`

The first one is the type of element we're creating, in this case an `<h1>` tag. This could also be another React component. Second is the properties list in the form of objects. Third argument is the content of the element used in the first argument.

We can create a React element to represent an h1 using React.createElement

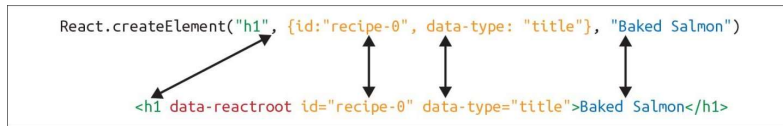
```
React.createElement("h1", null, "Baked Salmon")
```

When an element has attributes, they can be described with properties. Here is a sample of an HTML h1 tag that has id and data-type attributes.

```
React.createElement("h1", {id: "recipe-0", 'data-type': "title"}, "Baked Salmon")
```

Unit 2: ReactJS

Relationship between createElement and the DOM element



Note: data-reactroot will always appear as an attribute of the root element of your React component

React Component in brief:[In detail in L2 and L3]

A user control that has **code to represent visual interfaces and data**. An isolated piece of code which can be reused in one or the other module. Contains a root component in which other subcomponents are included. 2 types of components in React.js can be created.

- **Stateless Functional Component**

- Includes simple JavaScript functions and immutable properties, i.e., the value for properties cannot be changed.

```
function Demo(props) {
  return <h1> Welcome to REACT JS, {props.Name} </h1>;
}
```

- **Stateful Class Component**

- Classes which extend the Component class from React library. The class component must include the render method which returns HTML.

```
class Demo extends React.Component{
  render(){ return <h1>
    Welcome to REACT JS, {props.Name} </h1>; }
}
```

Calling/rendering the components:

ReactDOM: Contains the tools necessary to render React elements in the browser.

All the tools necessary to generate HTML from the virtual DOM are found in this library.

ReactDOM.render() is responsible for rendering a React component. The first

Unit 2: ReactJS

parameter is the component class name. Second parameter is the destination where the component is to be rendered.

```
ReactDOM.render(
  React.createElement(Demo, null, null),
  document.getElementById('root')
);
```

Coding Example 1: Serverless Hello world: Using react, displaying a simple page on the browser

```
<!DOCTYPE HTML>
<html>
<head>
  <script crossdomain
src="https://unpkg.com/react@16/umd/react.development.js"> </script>
  <script crossdomain src="https://unpkg.com/react-dom@16/umd/react-
dom.development.js"> </script>
  <script src="https://unpkg.com/babel-standalone@6.15.0/babel.min.js">
  </script>
</head>
<body>
  <div id="contents"></div><!-- this is where our component will appear -->
  <script type="text/babel"> // Important
    var contentNode = document.getElementById('contents');
    var component = <h1>Hello World!</h1>; // A simple JSX component
    ReactDOM.render(component, contentNode);
    // Render the component inside the content Node
  </script>
</body>
</html>
```

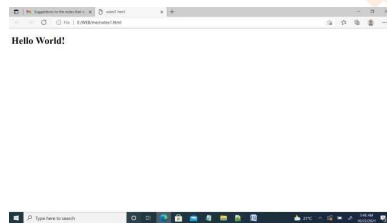
In the above code, `<script src="https://unpkg.com/babel-standalone@6.15.0/babel.min.js">`

Unit 2: ReactJS

`</script>`, this is the **babel library which is a JSX transformer**. Then, there is the type of the script that you specify in the wrapping `<script>` tags around the JSX ode. The browser-based JSX compiler looks for all inline scripts of type “text/babel” and compiles the contents into the corresponding JavaScript. The other two scripts, `react.js` and `react-dom.js`, are the core React libraries that handle react component creation and rendering

```
<script type="text/babel">
```

Screenshot of example code 1 output:

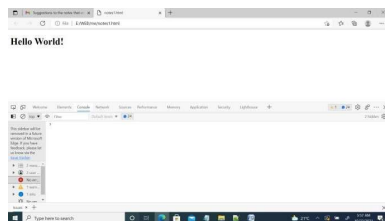


Coding Example 2: Multiple component calls/renders will render only the last one

In the above code, add these below lines and observe the output

```
ReactDOM.render(component, contentNode);  
ReactDOM.render(component, contentNode);  
ReactDOM.render(component, contentNode);  
ReactDOM.render(component, contentNode);
```

Screenshot of example code 1 output:



Unit 2: ReactJS

Including the JSX Code:

JSX uses a special syntax which allows you to mix HTML with JavaScript. JSX is a JavaScript XML used in React applications. An extension to JavaScript. Uses HTML syntax to create elements and components. Has tag name, attributes, and children. JSX compiles the code into pure JavaScript which can be understood by the browser.

Include the library:

```
<script src="https://unpkg.com/babel-standalone@6.15.0/babel.min.js"> </script>  
<script type="text/babel">  
    ReactDOM.render(<h1>Welcome to REACTJS</h1>,  
    document.getElementById('root')  
    );
```

Babel:

A JavaScript compiler that can translate markup or programming languages into JavaScript. Available for different conversions. React uses Babel to convert JSX into JavaScript.

Using React, print welcome to REACTJS on the web page. Use JSX and NonJSX both

Case 1: Using JSX Syntax

```
<html>  
<head>  
<script crossdomain src="https://unpkg.com/react@16/umd/react.development.js">  
</script>  
<script crossdomain src="https://unpkg.com/react-dom@16/umd/react-dom.development.js"> </script>  
<script src="https://unpkg.com/babel-standalone@6.15.0/babel.min.js"> </script>  
</head>  
<body>  
    <div id = "root"></div>  
    <script type = "text/babel">
```

Unit 2: ReactJS

```
ReactDOM.render(<h1>welcome</h1>,document.getElementById("root"))
</script>
</body>
</html>
```

Case 2: Using Non-JSX Syntax: small change in the above code

```
<html>
<head>
<script crossdomain src="https://unpkg.com/react@16/umd/react.development.js">
</script>
<script crossdomain src="https://unpkg.com/react-dom@16/umd/react-
dom.development.js"> </script>
<script src="https://unpkg.com/babel-standalone@6.15.0/babel.min.js"> </script>
</head>
<body>
<div id = "root"></div>
<script type = "text/babel">
ReactDOM.render(React.createElement("h1","null","welcome to
reactJS"),document.getElementById("root"))
</script>
</body>
</html>
```

References:

- i) [What is the use of React.createElement ? - GeeksforGeeks](#)
- ii) Textbooks from the syllabus copy

WEB TECHNOLOGIES

React JS – Components and Properties

Prof. Vinay Joshi and Prof. Sindhu R Pai
Department of Computer Science and Engineering

React.JS – Components and Properties Agenda

- Introduction to Components
- Types of components
- First Stateful component
- Parameterized components
- Components with child element
- Usage of props.child
- Introduction to Properties
- Validating and setting property values
- Transferring properties

React.JS – Components and Properties Introduction to Components

- Defines the **visuals and interactions** that make up what people see in app.
- Need for Components
 - There may be elements which are similar
 - A need to make a change will result in changes in multiple places
 - Similar to functions, if we could write code related to the element in one place, changes can be minimized
- Solution - Reusable piece of JavaScript code that output (via JSX) HTML elements



React.JS – Components and Properties Types of Components

- **Stateless Functional Component**
 - Includes simple JavaScript functions and immutable properties, i.e., the value for properties cannot be changed.
 - Use hooks to achieve functionality for making changes in properties using JS.
 - Used mainly for UI.
- **Stateful Class Component**
 - Classes which extend the Component class from React library. The class component must include the render method which returns HTML.



React.JS – Components and Properties

First Stateful Component



Creation of a component:

```
class HelloWorld extends React.Component
{
  render()
  {
    return <p>Hello, componentized world!</p>;
  }
}
```

Calling a component: Add the JSX in the render method with a element with the tag name as the Component name

```
ReactDOM.render( <HelloWorld/>,
  document.querySelector("#container")
);
```



React.JS – Components and Properties

Introduction to Properties(props)



- Properties are ways in which **React components can be customized**
- Properties to React components are same as what attributes are to HTML elements.
- **Props are like function arguments, you can send them into the component as attributes.**
- Their most basic use is in the form of attributes, in JSX.
- Are actually passed to a constructor and can be accessed by **this.props**
- Props are immutable
- camelCase in JSX

React.JS – Components and Properties

First Stateful Component



```
import React from 'react';
import ReactDOM from 'react-dom/client';
```

```
// Define the HelloWorld component
class HelloWorld extends React.Component {
  render() {
    return <p>Hello, componentized world!</p>;
  }
}
```

```
// Create a root for rendering
const root = ReactDOM.createRoot(document.querySelector("#container"));
```

```
// Render the HelloWorld component
root.render(<HelloWorld />);
```



React.JS – Components and Properties

Parameterized components



Accepting the properties in the component and passing these properties while calling the components.

Creation of a component:

```
class HelloWorld extends React.Component {
  render() { return <p>Hello, {this.props.greetTarget}</p>; }
}
```

Calling a component:

```
ReactDOM.render(
  <div>
    <HelloWorld greetTarget="Batman"/> <HelloWorld greetTarget="Iron Man"/>
  </div>,
  document.querySelector("#container")
);
```

We place it inside curly brackets - { }. In JSX, if you want something to get evaluated as an expression, you need to wrap that something inside curly brackets.



React.JS – Components and Properties

Parameterized components

```
import React from 'react';
import ReactDOM from 'react-dom/client';

// Define the HelloWorld component
class HelloWorld extends React.Component {
  render() {
    return <p>Hello, {this.props.greetTarget}!</p>;
  }
}

// Create a root for rendering
const root = ReactDOM.createRoot(document.querySelector("#container"));

// Render the HelloWorld components
root.render(
  <div>
    <HelloWorld greetTarget="Batman"/>
    <HelloWorld greetTarget="Iron Man"/>
  </div>
);
```

React.JS – Components and Properties

Components with Child elements

Your components can have children. The child elements in a component can be accessed by the **children property** accessed by **this.props.children**.

Creation of a component:

```
class Buttonify extends React.Component { render()
{ return( <div>
    <button type={this.props.behavior}>
    {this.props.children}</button>
  </div> );
}
}
```

Calling a component:

```
ReactDOM.render(
  <div>
    <Buttonify behavior="submit">SEND DATA</Buttonify>
  </div>, document.querySelector("#container")
);
```



React.JS – Components and Properties

Usage of props.children

- Components can have one element, multiple elements, or none at all. It's value is respectively a single child node, an array of child nodes or undefined.
- **A component with children is always identified by an opening tag and a closing tag.**
- Each child must be placed between these two tags
- **Example**
- **<MyComponent>foo</MyComponent>**
 - MyComponent has one children foo
- A special property that is passed to components automatically
- **Content between those tags is passed as a special prop: props.children**



React.JS – Components and Properties

Components with Child elements

```
import React from 'react';
import ReactDOM from 'react-dom/client';
```

```
class Buttonify extends React.Component {
  render() {
    return (
      <div>
        <button type={this.props.behavior}>
          {this.props.children}
        </button>
      </div>
    );
  }
}
```

// Define the Buttonify component

```
const root = ReactDOM.createRoot(document.querySelector("#container"));
root.render(
  <div>
    <Buttonify behavior="submit">SEND DATA</Buttonify>
  </div>
);
```

// Create a root for rendering
// Render the Buttonify component



React.JS – Components and Properties

Components with Child elements

Ensure your HTML file includes the container element:

```
<!DOCTYPE html>
<html>
<head>
  <title>React 18 App</title>
  <script src="https://unpkg.com/react@18/umd/react.development.js" crossorigin></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js" crossorigin></script>
  <script type="module" src="path-to-your-js-file.js"></script> <!-- Make sure to reference your
JS file here -->
</head>
<body>
  <div id="container"></div>
</body>
</html>
```

React.JS – Components and Properties

Validating and Setting the property values

```
class MyComponent extends React.Component {
  render() {
    return <div>{this.props.message}</div>;
  }
}

MyComponent.defaultProps = {
  message: 'Default message'
};
```

Notes

1. **Deprecated Method:** The `getDefaultProps` method was used in older versions of React but is deprecated in favor of **defaultProps**. It is not recommended to use `getDefaultProps` in new code.
2. **PropTypes:** Validating props with `PropTypes` is supported in both React 17 and React 18.



React.JS – Components and Properties

Validating and Setting the property values

- Validating the type of properties
 - Possible by providing any of the types in **React.PropTypes.***
- Setting the default property values
 - Can be overridden by properties passed in initialisation using **getDefaultProps()** method
 - Must return an object. The contents of that object are up to us



- Coding examples



React.JS – Components and Properties

Validating and Setting the property values

```
//Greeting.js
import React from 'react';

// Define the component
class Greeting extends
React.Component {
  render() {
    return <h1>Hello,
{this.props.name}</h1>;
  }
}

// Set default props
Greeting.defaultProps = {
  name: 'Guest'
};

export default Greeting;
```

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import Greeting from './Greeting'; // Assume the component is saved in Greeting.js

const root = ReactDOM.createRoot(document.querySelector("#container"));

root.render(
  <div>
    <Greeting name="Batman" />    {/* This will render: Hello, Batman! */}
    <Greeting />                  {/* This will render: Hello, Guest! */}
  </div>
);
```

React.JS – Components and Properties

Transferring Properties



- Sending properties down the structural tree while working with multiple components in a hierarchy
- **transferPropsTo():**
 - Passes all the properties of a parent component to a child component unless we explicitly set the value of a child component's property
- Coding examples

React.JS – Components and Properties

Transferring Properties



In modern React (including React 17 and 18), the **transferPropsTo** method is outdated and has been replaced by a more standardized approach to passing props through components. **Instead of transferPropsTo, you should use explicit prop passing.**

```
//Parent Component
import React from 'react';
import ChildComponent from './ChildComponent'; // Assume ChildComponent is in the same directory
```

```
class ParentComponent extends React.Component {
  render() {
    return (
      <div>
        <ChildComponent name="React" age={8} />
      </div>
    );
  }
}
```

```
export default ParentComponent;
```

React.JS – Components and Properties

Transferring Properties



```
//Child Component
import React from 'react';

class ChildComponent extends React.Component {
  render() {
    return (
      <div>
        <h1>Name: {this.props.name}</h1>
        <h2>Age: {this.props.age}</h2>
      </div>
    );
  }
}

export default ChildComponent;
```



THANK YOU

Vinay Joshi and Sindhu R Pai
Department of Computer Science and Engineering

vinayj@pes.edu
+91 9886703973

sindhurpai@pes.edu
+91 8277606459

Components and Properties

Components

It defines the **visuals and interactions** that make up what people see in app. Declares **how the view looks like, given the data**. When the data changes, if you are used to the jQuery way of doing things, you'd typically do some DOM manipulation. But the React doesn't do anything like that! The React library figures out how the new view looks, and just applies the changes between the old view and the new view. This makes the views consistent, predictable, easier to maintain, and simpler to understand.

Why Components?

There may be elements which are similar. There might be a need to make a change that will result in changes in multiple places. Similar to functions, if we could write code related to the element in one place, changes can be minimized. Solution is to have reusable piece of JavaScript code that output (via JSX) HTML elements

Types of Components

1. Stateless Functional Component – [Will be discussed later]

- Includes simple JavaScript functions and immutable properties, i.e., the value for properties cannot be changed.
- Use hooks to achieve functionality for making changes in properties using JS.
- Used mainly for UI.

2. Stateful Class Component

- Classes which extend the Component class from React library. The class component must include the render method which returns HTML.

First Stateful component

Creation of a component:

```
class HelloWorld extends React.Component
{
  render()
  {
    return <p>Hello, componentized world!</p>;
  }
}
```

The render() method is something that the React calls when it needs to display the component. There are other methods with special meaning to React that can be implemented, called the Lifecycle functions, which provide hooks into various stages of the component formation and events. This will be discussed later. But render() is one that must be present; otherwise you'll have a component that has no screen presence.

Calling/Rendering a component:

Case 1: Add the JSX in the render method with a element with the tag name as the Component name

```
ReactDOM.render( <HelloWorld>,
  document.querySelector("#container")
);
```

Case 2: Add the NON-JSX in the render method with a element using React.createElement

```
ReactDOM.render( React.createElement("HelloWorld",null, null),
  document.querySelector("#container")
);
```

Parameterized components:

Passing the attributes to the components during rendering and using these attributes as properties inside the component.

Properties

Properties are ways in which React components can be customized. Immutable and same as what attributes in HTML elements. Props are arguments passed into React components and are passed via HTML attributes. 2 steps to add properties to components

Unit 2: ReactJS

- Make the function of your component read the props from the props parameter
Place the props inside curly brackets – { }. In JSX, if you want something to get evaluated as an expression, you need to wrap that something inside curly brackets.
If you don't do that, the raw text gets printed out.

```
return <h1>Hello, {this.props.greetingtarget}</h1>
```

- Modify the Component Call :** When rendering the component, add the prop to the component using the attribute

```
<Helloworld greetingtarget = "sindhu" />
```

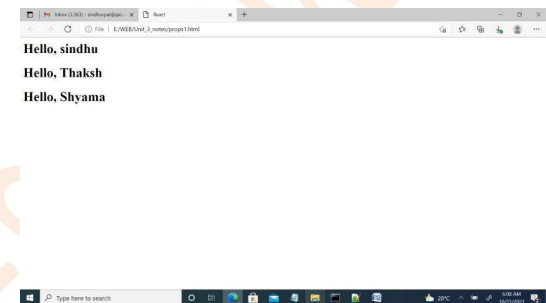
Coding example 1: Usage of this.props

```
<html>
  <head>
    <title>React</title>
    <script crossdomain
src="https://unpkg.com/react@16/umd/react.development.js"> </script>
    <script crossdomain src="https://unpkg.com/react-dom@16/umd/react-
dom.development.js"> </script>
    <script src="https://unpkg.com/babel-standalone@6.15.0/babel.min.js">
  </script>
  </head>
  <body>
    <div id="container"></div>
    <script type="text/babel">
      var dest = document.querySelector("#container");
      class Helloworld extends React.Component {
        render(){
          return <h1>Hello, {this.props.greetingtarget}</h1>
        }
      }
```

Unit 2: ReactJS

```
    }
    ReactDOM.render(
      <div>
        <Helloworld greetingtarget = "sindhu" />
        <Helloworld greetingtarget = "Thaksh" />
        <Helloworld greetingtarget = "Shyama" />
      </div>,
      document.getElementById("container")
    );
  </script>
</body>
</html>
```

Output:



Usage of this.props.children

A special property that is passed to components automatically. It is used to display the data between the opening and closing JSX tags when invoking a component. Can have one element, multiple elements, or none at all. It's value will be respectively a single child node, an array of child nodes or undefined

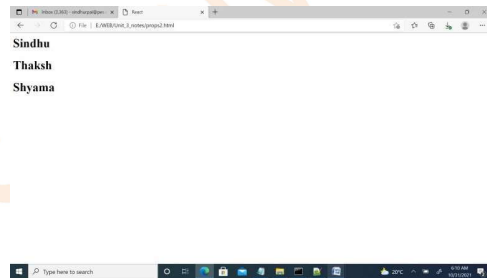
Coding example 2:

```
<script type="text/babel">
```

Unit 2: ReactJS

```
class Helloworld extends React.Component {  
  render(){  
    return <h1>{this.props.children}</h1>  
  }  
}  
ReactDOM.render(  
  <div>  
    <Helloworld>Sindhu</Helloworld>  
    <Helloworld>Thaksh</Helloworld>  
    <Helloworld>Shyama</Helloworld>  
  </div>,  
  document.querySelector("#container")  
)  
</script>
```

Output:



Validating the property values

The properties being passed to component can be validated against a specification. This specification is supplied in the form of a static object called `propTypes` in the class, with the name of the property as the key and the validator as the value, which is one of the many constants exported by `React.PropTypes`, for example,

Unit 2: ReactJS

React.PropTypes.string

```
Helloworld.propTypes = { greetingtarget: React.PropTypes.string.isRequired,  
  greetingtarget_id: React.PropTypes.number  
};
```

Property validation is checked only in development mode, and a warning is shown in the console when any validation fails. Since we are in an early stage in the development of the application, we expect more changes to the properties. You can also default the property values when the parent does not supply the value. For example, if you want the `greeting_target` to be defaulted to something else, rather than show an empty string, you can do this: `Helloworld.defaultProps = { greetingtarget: '-- no title --', };`

Few Points to think:

- Can we have nested components?
 - Can we provide style to components?
 - How to send properties from one component to another?
 - What are the different methods under life cycle of components?
- Refer to the further notes for more details on the above points

WEB TECHNOLOGIES

React JS – Styling the Components and Complex Components

Prof. Vinay Joshi and Prof. Sindhu R Pai
Department of Computer Science and Engineering

React.JS – Styling the Components

Agenda

- Introduction
- Inline CSS
- Usage of props.children
- Demo code

React.JS – Styling the Components

Introduction

- Different ways to style React Components
 - **Inline CSS** - using style attribute
 - CSS in JS - Using Libraries JSS and Styled Components
 - CSS modules – css loader and Sass & SCSS
 - Stylable

React.JS – Styling the Components

Inline CSS

- To style an element with the inline **style attribute**, the value must be a JavaScript object
- In JSX, JavaScript expressions are written inside curly braces and JavaScript objects also use curly braces
- Example:

```
class MyHeader extends React.Component {  
  render() { return (  
    <h1 style={{color: "red"}}>Hello Style!</h1>  
    <p>Add a little style!</p> ); }  
}
```
- camelCased Property Names

```
<h1 style={{backgroundColor: "lightblue"}}>Hello Style!</h1>
```



React.JS – Styling the Components

Inline CSS

```
import React from 'react';

class MyHeader extends React.Component {
  render() {
    return (
      <div>
        <h1 style={{ color: 'red', backgroundColor: 'lightblue' }}>
          Hello Style!
        </h1>
        <p>Add a little style!</p>
      </div>
    );
  }
}
export default MyHeader;
```

React.JS – Styling the Components

Inline CSS continued..

```
import React from 'react';

class Letter extends React.Component {
  render() {
    // Define the style object
    const letterStyle = {
      padding: 10,
      margin: 10,
      backgroundColor: "#ffde00",
      color: "#333",
      display: "inline-block",
      fontFamily: "monospace",
      fontSize: 32,
      textAlign: "center"
    };

    return (
      <div style={letterStyle}>
        {this.props.children}
      </div>
    );
  }
}
export default Letter;
```



React.JS – Styling the Components

Inline CSS continued..

- Creation of javascript object which contains the css properties and values in the form of key value pair
- Passing this as a value to style attribute
- Example:

```
class Letter extends React.Component {
  render() {var letterStyle = { padding: 10, margin: 10, backgroundColor: "#ffde00",
    color: "#333", display: "inline-block", fontFamily: "monospace",
    fontSize: 32, textAlign: "center"
  };
  return(
    <div style={letterStyle}>
      {this.props.children}
    </div>    ); }
}
```



React.JS – Styling the Components

Demo

- Create the following letters using react components
- Additionally, apply styles to them differently by passing styling properties



React.JS – Complex Components

Agenda

- Introduction
- Approach followed
- Complex Component creation
- Sub component creation
- Properties from parent to sub component
- Modified parent and sub components
- Transferring properties



THANK YOU

Vinay Joshi and Sindhu R Pai

Department of Computer Science and Engineering

vinayj@pes.edu
+91 80 2672 6622

sindhurpai@pes.edu
+91 8277606459

UNIT2: ReactJS

Styling the components and Complex Components

Styling the components

There are different ways to style the components in React.

- Inline CSS – using the style attribute
- CSS in JS - Using Libraries JSS and Styled Components
- CSS Modules - css loader and Sass & SCSS
- Stylable

Inline CSS in Detail

Consider the below code to provide red color to the contents of h1. But this is not a react way of providing the styling to the components.

```
render()
{
  return (<h1 style = {color:"red"}>welcome to styling to components</h1>)
}
```

We create objects of style and render it inside the components in style attribute using the React technique. Let us understand this through an example code.

```
<body>
<div id = "root"></div>
<script type = "text/babel">
  class Letter extends React.Component{
    render() {
      //Object letterstyle contains all key-value pairs of styling to be applied to letter
      var letterstyle = {
        marginRight: "10px", // observe this comma
        textAlign: "center", backgroundColor:"blue", //observe the camelcase
        color:"olive", padding: "20px", display:"inline"
      }
      return <h1 style = {letterstyle}> {this.props.children}</h1>
    }
  }
//Setting the style attribute to refer to the object
```

```
}
}

ReactDOM.render(<div><Letter>A</Letter>
  <Letter>E</Letter>
  <Letter>I</Letter>
  <Letter>O</Letter>
  <Letter>U</Letter> </div>
  ,document.getElementById("root"))

</script>
<body>

If we execute the above code, All letters will have the same background color. To avoid
this, background color must be sent during the component rendering.

<script type = "text/babel">
  class Letter extends React.Component{
    render() {
      //Object letterstyle contains all key-value pairs of styling to be applied to letter
      var letterstyle = {
        marginRight: "10px", // observe this comma
        textAlign: "center", backgroundColor: this.props.bgcolor,
        color:"olive", padding: "20px"
      }
      return <h1 style = {letterstyle}> {this.props.children}</h1>
    }
  }

  ReactDOM.render(<div><Letter bgcolor = "pink"> A</Letter>
    <Letter bgcolor = "red">E</Letter>
    <Letter bgcolor = "#0000FF">I</Letter>
    <Letter bgcolor = "#BBEE00">O</Letter>
    <Letter bgcolor = "#EE00BB">U</Letter> </div>
    ,document.getElementById("root"))

</script>
```

Output of above code is as below:



Advantage of creation of style object is , it provides flexibility to add more styling properties as and when required.

Complex Components

It is nothing but **building the component that uses other user defined components**. Also known as **component composition**. **React** lets you split the UI into smaller independent pieces so that you can reason about each piece in isolation. Using components rather than building the UI in a monolithic fashion also encourages reuse.. A component takes inputs (called properties) and its output is the rendered UI of the component. We will put together fine-grained components to build a larger UI.

Approach followed to build complex components:

- Identify the major visual elements
- Breaking them into individual components

Consider this example:

